



An experimental evaluation of weightless neural networks for multi-class classification

Massimo De Gregorio^a, Maurizio Giordano^{b,*}

^a Istituto di Scienze Applicate e Sistemi Intelligenti CNR, Via Campi Flegrei, 34, 80078 Pozzuoli, Naples, Italy

^b Istituto di Calcolo e Reti ad Alte Prestazioni CNR, Via Pietro Castellino, 111, 80131 Naples, Italy



ARTICLE INFO

Article history:

Received 15 September 2017

Received in revised form 6 July 2018

Accepted 26 July 2018

Available online 13 August 2018

Keywords:

Weightless neural network

WiSARD

Machine learning

ABSTRACT

WiSARD belongs to the class of *weightless neural networks*, and it is based on a neural model which uses lookup tables to store the function computed by each neuron rather than storing it in weights of neuron connections. WiSARD is characterised by a simple implementation and a fast learning phase due to one-way RAM access/lookup mechanism. WiSARD was originally conceived as a pattern recognition device mainly focusing on image processing. In this work we present a multi-class classification method in machine learning domain based on WiSARD, called *WiSARD Classifier*. The method uses the same binary encoding scheme to transform multivariable data in the domain of real numbers into binary patterns which are the input to WiSARD. The main contribution of this work is an extensive experimental evaluation of WiSARD's classification capability in comparison to methods from the state-of-the-art. For the purpose we conducted many experiments applying nine well known machine learning methods (including the *WiSARD Classifier*) to seventy classification problems. Cross-validation accuracies were collected and compared by means of a statistical analysis based on nonparametric tests (Friedman, Friedman Aligned Rank, and Quade test) to prove how the *WiSARD Classifier* is very close in performance to the best methods available in most popular machine learning libraries.

© 2018 Elsevier B.V. All rights reserved.

1. Introduction

The n -tuple recognition method invented by Bledsoe & Brown-ing (1959) [1] is also known in literature as “RAMnet” or “weightless neural network” (WNN). The basic computation unit of this method is the n -tuple neuron: a RAM with 2^n memory cells (and address lines); n -tuples, which are sequences of n bits sampled from input data, are used to access (stimulate) memory cells to write (learn) or read (analyze) neuron contents. This is why n -tuple recognition methods implemented with RAM neurons are also called in literature “weightless neural networks”. Exhaustive surveys of WNN models are discussed in [2,3].

WNNs have many similarities with another class of models referred in literature as “associative neuron networks”, like the Kanerva model [4] and CMAC [5]. In both WNNs and associative neuron networks an input vector of values is mapped to a set of memory cells: in the weightless models the information stored in memory cells plays the same role of weights in multi-layer

neural networks. While, in the latter nonlinearity relies on the sigmoid/threshold transfer function, in the former models nonlinearity is implemented by the input-to-memories mapping.

In terms of functionality, a weighted-sum-and-threshold artificial neuron having n inputs can be exactly imitated by an n -tuple RAM node, as demonstrated by [6]. However, the way learning is performed in weighted versus weightless models differs in the following: the functionality of a neuron can be modified by changes in the weights in the former model, and by changes in the RAM contents in the latter one. From the literature [1,7] it is widely accepted that WNNs belong to the class of neural network models, like the weighted counterparts.

Originally conceived as a pattern recognition device, WiSARD¹ is a WNN model of computing which has been widely used in literature in the image processing domain. A main advantage of WiSARD is the fast feed-forward and incremental learning scheme in alternative to the well known backpropagation algorithm for training of neural networks. Backpropagation, which is generally based on gradient search techniques, requires many iterations to converge; in addition, it works adjusting weights to find local minima of an

* Corresponding author.

E-mail addresses: massimo.degorgio@cnr.it (M. De Gregorio), maurizio.giordano@cnr.it (M. Giordano).

¹ Wilkes, Stonham and Aleksander Recognition Device.

error surface by considering all training inputs. Differently from backpropagation, the WiSARD learning mechanism is appropriate for on-line learning since it allows incremental learning at low computational costs.

In past years different WiSARD applications have been proposed as multi-class classifiers in different domains, ranging from text categorisation [8], to HIV-1 subtypes classification [9], from language POS-tagging [10] to WiFi signal pattern categorisation [11]. All the cited works are specialized applications of weightless systems to specific problem domains. As such, in these works the adopted WNN systems have been tuned and configured to optimally behave in the intended target ML domains. In this work we propose *WiSARD Classifier*, that is a supervised method for data classification in ML domain based on learning and classification capabilities of WiSARD. The proposal of a WiSARD-based method for data recognition/classification is not new, since in the past similar approaches have been proposed in several data domains (mainly in image processing). Nevertheless *WiSARD Classifier* can be regarded, as far as we know, the first domain-independent classification method for ML. In fact, it uses the same binary encoding scheme to transform multivariable numeric data samples into binary patterns which are learned-then-classified by a WiSARD neural network. *WiSARD Classifier*'s implementation² supports well-known input data formats, such as LibSVM, ARFF and CSV formats.

The main contribution of this work is to give experimental evidence of the good performance of our method, by showing how, in the average, *WiSARD Classifier* shows performances comparable to state-of-the-art ML methods found in literature. This statement is validated by a detailed nonparametric statistical analysis we carried out on a set of experiments consisting in running different ML classifiers, including *WiSARD Classifier*, on a set of ML datasets downloaded from the KEEL-dataset repository.³

The paper is so organized: Section 2 introduces the WiSARD model with a formalization of its architecture, the training and the classification function, together with a discussion on some model extensions. In Section 3, some related learning models which have similarities to WiSARD are discussed. Section 4 discusses the methodology of *WiSARD Classifier* by describing the data transformation scheme, which is required to feed the WiSARD network with proper inputs, as well as the algorithm for resolving classification ties. Section 5 reports results of applying several ML classifiers (including our proposal) on the large KEEL-dataset repository. In Sections 6–8 we report a nonparametric statistical analysis on experimental results that demonstrates the good performance of the proposed method with respect to state-of-the-art ML classifiers. In Section 9, the computational complexity and timing estimates of *WiSARD Classifier* are discussed in comparison to other methods from the state-of-the-art. Section 10 reports some concluding remarks.

2. WiSARD computational model

The definition of WiSARD can be found in [12,13]. In what follows, we give a formal definition of WiSARD model architecture, training and classification functions (see Sections 2.1–2.3). In addition, we formalize the generalization property of WiSARD and we discuss performance dependency on hyper-parameters (see Section 2.4). At the end of the section, the model extension used as core learning mechanism in our method is described (see Section 2.5). This extension was required to support the algorithm (introduced in Section 4.2) we used to improve classification capability.

2.1. The formal model

Let us consider a bit vector $\mathbf{u} = \{u_1, u_2, \dots, u_L\}$ of size $L = N \times n$, with N and n integers. The bit vector is called *retina*. A set of n distinct bit locations randomly selected from $\mathbf{u}$ forms an n -tuple. The collection of N different n -tuples (which covers the entire retina) is called *input mapping function*, and can be so defined:

$$\eta_i(j) = \text{the } j\text{th bit location in the } i\text{th } n\text{-tuple, for } i \in \{1, \dots, N\}, \\ j \in \{1, \dots, n\}.$$

With this definition, through the *input mapping function* we have that:

$$u_{\eta_i(j)} = \text{the bit at } j\text{th location in the } i\text{th } n\text{-tuple extracted from the bit vector } \mathbf{u}.$$

The WiSARD n -tuple classifier is formed by as many *discriminators* as the number of classes it has to discriminate between. Each discriminator consists of a set of N RAMs that, during the training phase, learn the occurrences of n -tuples extracted from the bit patterns $\mathbf{u}$ loaded on the retina.

Fig. 1 describes the discriminator's structure and how RAM memory cells are accessed during training and classification of WiSARD.

In the model, n -tuples selected from the input binary vector are regarded as the “features” of the input pattern to be recognized. In literature [14] has been argued that the randomness of feature extraction makes WiSARD more sensitive to detect global features than an ordered map which makes a single layer system sensitive to detect local features.

When bit vector size is larger (in magnitude order) than n , due to combinatorial explosion, it is hard to fully formalize and measure tuple correlations, as the analysis in [15] proved.

Let the θ function be:

$$\theta(x) = \begin{cases} x & \text{if } 0 \leq x \leq 1 \\ 1 & \text{if } x > 1 \end{cases} \quad (1)$$

and let $\mathbf{v}$ be a pattern⁴ in the Training Set for the class c , named TS_c . Each RAM_i in the network of N RAMs is uniquely associated to the i th n -tuple, where RAM_i has 2^n cells with addresses $\alpha \in [0, 2^n - 1]$. An address of RAM_i can be generated starting from the pattern $\mathbf{v}$ in the following way:

$$\alpha_i(\mathbf{v}) = \sum_{j=1}^n v_{\eta_i(j)} 2^{j-1} \quad (2)$$

that is, the mapping $\eta_i(j)$ selects the bits of the i th n -tuple from the pattern $\mathbf{v}$, whereas bit locations are randomly placed in the bit vector. The address $\alpha_i(\mathbf{v})$ is used to stimulate (to access) one memory cell of RAM_i which is associated to the i th n -tuple, either in writing (training) or reading (classification) mode.

2.2. Training

Let $m_{c,i,\alpha}$ be the memory cell with address α in the RAM_i belonging to the discriminator of the class c . During the training phase, the memory cell $m_{c,i,\alpha}$ (initialized with zero contents) is updated with the formula:

$$m_{c,i,\alpha} = \theta \left(\sum_{\mathbf{v} \in TS_c} \delta_{\alpha, \alpha_i(\mathbf{v})} \right) \quad (3)$$

² Available for download at <https://github.com/giordamaug/WisardClassifier-C-vectors>.

³ <http://sci2s.ugr.es/keel/datasets.php>.

⁴ In the rest of the paper we denote with $\mathbf{v}$ patterns used for training, and with $\mathbf{u}$ patterns to be classified.

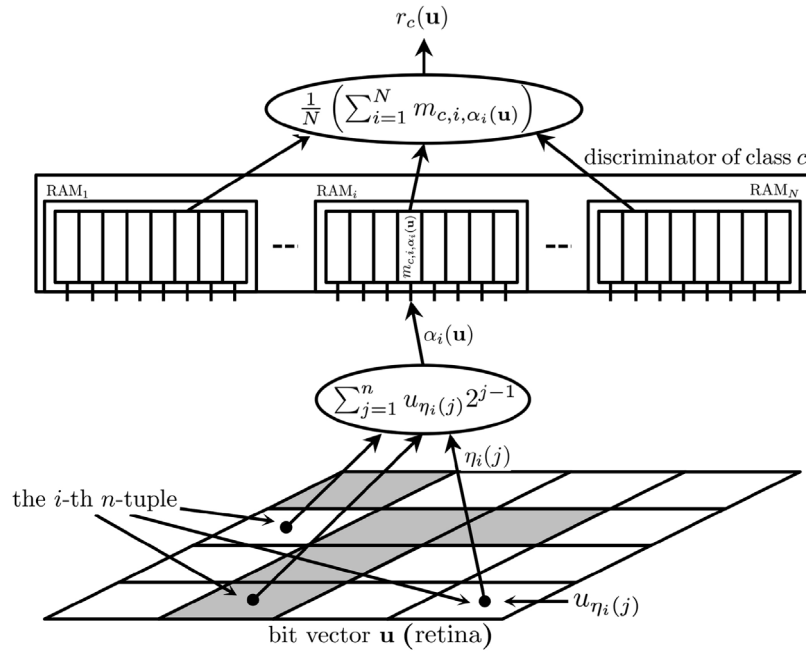


Fig. 1. WiSARD discriminator model.

where δ is the Kronecker delta function:

$$\delta_{\alpha, \alpha_i(\mathbf{v})} = \begin{cases} 1 & \text{if } \alpha = \alpha_i(\mathbf{v}) \\ 0 & \text{elsewhere} \end{cases} \quad (4)$$

As the formula of Eq. (3) shows, the training procedure of RAM_i in WiSARD (and in RAMnet models in general) is implemented by summing the write effects at each memory cell $m_{c,i,\alpha}$ of the RAM stimulated by the i th n -tuple extracted from the training pattern $\mathbf{v}$. Moreover, the order by which patterns are input to the network during training is irrelevant with respect to the final knowledge stored after the training.

It should be noted that the θ function guarantees that if at least one n -tuple is used to access $m_{c,i,\alpha}$ then its content is updated to 1 (further occurrences of the same n -tuple cause the same update, thus the memory content remains unchanged). Otherwise, if no n -tuple from the training patterns generates the generic address α then the memory cell $m_{c,i,\alpha}$ is not changed (remaining zeroed).

Such a definition of training procedure is the main difference of the WiSARD model with respect to adaptive learning of typical neural networks such as multi-layer perceptron and deep learning. In general, this is also the main advantage of RAMnet models that provide faster learning procedures.

2.3. Classification

Once training of WiSARD is finished, we can classify an unseen pattern $\mathbf{u}$. The WiSARD n -tuple recognizer operates simply as follows:

A pattern $\mathbf{u}$ is classified as belonging to the class c if it has the most features in common with at least one training pattern of that class.

Formally, pattern $\mathbf{u}$ is assigned to class c , if c is the class whose discriminator gives the highest output, that is:

$$\arg\max_c \left(\sum_{i=1}^N m_{c,i,\alpha_i(\mathbf{u})} \right). \quad (5)$$

On the other hand, the output (*response*) of the discriminator associated to class c on input pattern $\mathbf{u}$ is given by:

$$r_c(\mathbf{u}) = \frac{1}{N} \left(\sum_{i=1}^N m_{c,i,\alpha_i(\mathbf{u})} \right). \quad (6)$$

The discriminator's response ranges in the real interval $[0, 1]$. Responses of a set of discriminators trained on K classes of patterns, form the components of the output *response vector*:

$$\mathbf{r}(\mathbf{u}) = \{r_{c_1}(\mathbf{u}), \dots, r_{c_K}(\mathbf{u})\}. \quad (7)$$

2.4. Generalization

The *generalization* property of a WiSARD is the capability of producing an appropriate output in response to an input binary vector not included in the training set. It is usually desirable for the WiSARD to generate similar outputs for similar input vectors.

The WiSARD can be generalized to a function interpolator [16] by considering the vector $\mathbf{m}_{i,\alpha}$ stored at each address α in each RAM_i across all discriminators. Training is accomplished by assigning these vectors to the averages:

$$\mathbf{m}_{i,\alpha} = \frac{\sum_{\mathbf{v} \in TS} \mathbf{Y}(\mathbf{v}) \delta_{\alpha, \alpha(\mathbf{v})}}{\sum_{\mathbf{v} \in TS} \delta_{\alpha, \alpha(\mathbf{v})}} \quad (8)$$

where $\mathbf{Y}(\mathbf{v})$ is the desired output vector for input pattern $\mathbf{v}$, which varies in the entire training set $TS = TS_{c_1} \cup \dots \cup TS_{c_K}$.

In [17] it is showed, with a probabilistic treatment, how the WiSARD discriminators have the property of generalizing provided that the training sets for each class of patterns are sufficiently distant (in term of Hamming distance). In what follows we re-propose the same probabilistic formalization of generalization in WiSARD to show how this model depends on two parameters: the tuple dimension n , and the pattern size L .

Let us consider the unseen pattern $\mathbf{u}$ and a training pattern $\mathbf{v}_j \in TS_c$ (for the class c of patterns). The *distance vector* is defined as:

$$\mathbf{d}(\mathbf{u}, \mathbf{v}) = \mathbf{u} \oplus \mathbf{v} \quad (9)$$

where the $\oplus$ operator calculates the XOR of bit components of the vector arguments. The *Hamming distance* is defined as:

$$H(\mathbf{u}, \mathbf{v}) = \sum_{l=1}^L d_l \quad (10)$$

Remind that, according to the WiSARD model definition, the same n -tuple of bits randomly chosen from the vector components of $\mathbf{u}$ and $\mathbf{v}_j$ are used to stimulate the same RAM in the RAM set of a discriminator.

Now, let us assume $H(\mathbf{u}, \mathbf{v}) > 0$, that is some bits in the distance vector between the two patterns are 1s. Then, the probability $p_i(\mathbf{u}, \mathbf{v})$ that the RAM_i , trained on the pattern $\mathbf{v}$, outputs a 1 upon classification of pattern $\mathbf{u}$ is the probability that none of the 1s of the distance vector $\mathbf{u} \oplus \mathbf{v}$ occurs at RAM_i : it is equivalent to the probability that all 1s occur at the other RAMs (which are $N - 1$), e.g. among the $(N - 1)n$ bits different from those associated to RAM_i . The probability $p_i(\mathbf{u}, \mathbf{v})$ is the fraction of the number of favorable events over the number of possible events:

$$p_i(\mathbf{u}, \mathbf{v}) = \frac{\binom{(N-1)n}{H(\mathbf{u}, \mathbf{v})}}{\binom{Nn}{H(\mathbf{u}, \mathbf{v})}} = \frac{(Nn - H(\mathbf{u}, \mathbf{v}))(Nn - H(\mathbf{u}, \mathbf{v}) - 1) \dots (Nn - H(\mathbf{u}, \mathbf{v}) - n + 1)}{(Nn)(Nn - 1) \dots (Nn - n + 1)} \quad (11)$$

The probability that RAM_i outputs a 1 on pattern $\mathbf{u}$ after being trained on all patterns $\mathbf{v} \in TS$, is given by:

$$p_i(\mathbf{u}) = 1 - \prod_{\mathbf{v} \in TS} q_i(\mathbf{u}, \mathbf{v}) \quad (12)$$

where $q_i(\mathbf{u}, \mathbf{v}) = 1 - p_i(\mathbf{u}, \mathbf{v})$. Of course, the probability that RAM_i outputs a 0 on pattern $\mathbf{u}$ is:

$$q_i(\mathbf{u}) = 1 - p_i(\mathbf{u}) = \prod_{\mathbf{v} \in TS} q_i(\mathbf{u}, \mathbf{v}).$$

Considering that the discriminator for the class c is formed by N RAMs, the probability of generating m ($0 \leq m \leq N$) 1s follows a binomial distribution:

$$P(\mathbf{u}) = \binom{N}{m} p_i^m(\mathbf{u}) q_i^{N-m}(\mathbf{u}). \quad (13)$$

The closer, in term of Hamming distance, is $\mathbf{u}$ to TS_c the higher is $P(\mathbf{u})$ and, whenever $\mathbf{u}$ is an element of the training set, $P(\mathbf{u})$ will be 1. This is true if TS is well defined in terms of recurrent pattern features and the Hamming distance between training patterns is low [14]. In other words, generalization is influenced adversely by within pattern variability and influenced favorably by among pattern variability. This is referred by Steck as “the effect of look-alikes” in [18].

In order to avoid degradation effect on the generalization capabilities of such systems, there are some actions that can be easily carried out on the training set. These actions are very well introduced and explained in [14].

The dependency between n and TS has been analyzed by Bledsoe in [19] and by Ullmann in [20]. They both show how the performance of n -tuple classifiers improves with n but with the agreement of increasing the expertise too (TS cardinality). However, Ullmann shows that, once fixed TS , there exists a maximum value of n over which the performance degrades. Because we can never increase as much as we wish the TS cardinality, there will be always a certain number of rarely occurring patterns on which the system fails to recognize. Luckily, this weakness is even manifest in all other recognition systems [20]. In [13], Aleksander shows

how discrimination power is very dependent on n . In particular, the system is very precise and little sensitive for n small (*overgeneralisation*) and, *vice versa*, very sensitive and little precise for n big (*undergeneralisation*) [14].

2.5. Model extension

DRASiW [21] is a variant of WiSARD model in which the θ function is extended in the following way:

$$\theta_\beta(x) = \begin{cases} x & \text{if } 0 \leq x \leq \beta \\ \beta & \text{if } x > \beta \end{cases} \quad (14)$$

that is, the WiSARD is extended in such a way to store in memory cells $m_{c,i,\alpha}$ a counter of occurrences of the same n -tuple in the training patterns, with a saturation value represented by β .

With this extension, we only need to change the memory content update procedure with the following:

$$m_{c,i,\alpha} = \theta_\beta \left(\sum_{\mathbf{v} \in TS_c} \delta_{\alpha, \alpha_i(\mathbf{v})} \right) \quad (15)$$

while discriminator response and class detection function (see Eqs. (5) and (6)) are unchanged.

The possibility of storing frequency counters of n -tuple occurrences during training allows DRASiW to work in a reverse order: frequencies stored in RAMs can be used to produce a histogram representation of n -tuples extracted from the training examples, or, in other words, a pictorial prototype of the function interpolated by each discriminator based on training examples. These prototypes are a sort of learned knowledge of RAM networks dumped in a data structure, called “mental image” (see [21]). The possibility of extracting and processing internal states of DRASiW memories is an interesting property which has been studied in [22,23] and explored in several applications [24,25].

3. Related models

Among the models related to the WiSARD we hereafter discuss: the Kanerva’s sparse distributed memory model [4], the cerebellar model arithmetic computer [5], the radial basis function networks [26], and the single-layer lookup perceptron [27].

3.1. Kanerva’s SDM

The Kanerva’s sparse distributed memory model [4] was proposed mainly as an associative memory, although it has also been used for classification problems and function interpolation. Instead of n -tuples extracted from the input bit vectors $\mathbf{v} = \{v_1, \dots, v_m\}$, in the Kanerva’s model a set of N bit vectors randomly chosen from a uniform distribution are used as *centers* ξ_i of hyper-sphere radial basis functions ϕ_i of binary vectors, such that:

$$\phi_i(\mathbf{v}, r) = \begin{cases} 1 & \text{if } H(\xi_i, \mathbf{v}) \leq r \\ 0 & \text{if } H(\xi_i, \mathbf{v}) > r \end{cases} \quad (16)$$

where $H(\xi_i, \mathbf{v})$ is the Hamming distance between the pattern $\mathbf{v}$ and the center ξ_i , and r is a properly chosen radius.

A memory space of cells (memory vector) with as many components as the function we want to approximate, is associated to each ξ_i . Like WiSARD, the training phase of the Kanerva’s SDM is incremental: each memory cell stores an averaged summation of the contributions of each training pattern, according to the formula of Eq. (17), where $\mathbf{Y}(\mathbf{v})$ is the desired output for input pattern $\mathbf{v}$. As we can see, Eq. (17) is the same memory update procedure of WiSARD (see Eq. (8)) with the only difference that Kronecker deltas are

replaced by radial functions. The network response to test pattern $\mathbf{u}$ is given by Eq. (18).

$$\mathbf{m}_i = \frac{\sum_{\mathbf{v} \in TS} \mathbf{Y}(\mathbf{v}) \phi_i(\mathbf{v}, r)}{\sum_{\mathbf{v} \in TS} \phi_i(\mathbf{v}, r)}, \quad (17)$$

$$\mathbf{r}(\mathbf{u}) = \frac{\sum_{i=1}^N \mathbf{m}_i \phi_i(\mathbf{u}, r)}{\sum_{i=1}^N \phi_i(\mathbf{u}, r)}. \quad (18)$$

In case of classification, the $\mathbf{Y}(\mathbf{v})$ is the indicator function, that is a vector with as many components as the number of classes to be detected, where the component $Y_c(\mathbf{v}) = 1$ if training pattern $\mathbf{v}$ belongs to class c , while the other components of $\mathbf{Y}$ are zeros.

3.2. Cerebellar model arithmetic computer (CMAC)

The CMAC is a neural network proposed by Albus in 1971 [5] as a learning method for artificial art movements in robot controlling system. CMAC is a single-layer neural network belonging to the class of associative neural networks, in which only a small subset of the network neurons influences the output, and that subset is determined by the input to the network.

The input is a vector with L components in a given space $D \in \mathbb{R}^L$. CMAC maps each input vector to a set of C points into a large conceptual memory: input vectors which are “close” (with respect to some metric) are mapped to C points of the conceptual memory which have several overlaps, while distant vectors are mapped to C points with no overlap. The associative mapping between the input components and the C points assures local generalization: similar inputs produce similar outputs while distant inputs produce nearly independent outputs. In addition, the associative mapping reduces the input space dimension from L to M , where M is the number of possible values of C points in the conceptual memory space.

Each of the M values for C points are then biunivocally and randomly mapped to an actual memory which is a single array of M memory cells (one RAM) used to store weights. While in WiSARD the input vector activates one memory cell in all RAMs of a discriminator, in CMAC the input vector identifies a set of C points. The C points are used as addresses of RAM cells whose contents (weights) are used as coefficients of a linear combination to produce the output. CMAC systems may have a single or multiple output nodes. In case of a single output node, the scalar response is:

$$r(\mathbf{u}) = \sum_{i=1}^M w_i \delta_{\alpha_i, \alpha_g(\mathbf{u})} \quad (19)$$

where $g(\mathbf{u})$ is the associative mapping of input vectors to C points, and $\alpha_{g(\mathbf{u})}$ is the address that the conceptual point activates in the actual memory (the single RAM). As we can see, the response of CMAC is very similar to the WiSARD since only a subset of memory cells is activated and contributes to the output, although the way the input vector $\mathbf{u}$ is mapped to the C points is different.

CMAC is trained in supervised mode based on a training dataset of vectors for which the expected network output is known. During training, weights stored in the actual memory are adjusted using the least mean squares algorithm. This is equivalent to a gradient search of a surface which is quadratic and therefore less computational costly with respect to general backpropagation scheme.

3.3. Radial basis function networks

Radial basis function (RBF) networks [26] are three layer neural networks: the input layer is a set on L nodes accepting the components of a L -dimensional vector $\mathbf{u}$. The input nodes are fully connected to a hidden layer with N nodes, whose activation functions are non-linear radial basis functions. The outputs of the

hidden-layer nodes is linearly combined in a output-layer composed of k nodes, each one providing the response $r_c(\mathbf{u})$ of the network with respect to the class C . Therefore the output of the network is a vector function of the input vector, with components given by the formula of Eq. (20), where ξ_i is the center vector for neuron i and $w_{c,i}$ is the weight of neuron i in the linear output neuron c .

$$r_c(\mathbf{u}) = \sum_{i=1}^N w_{c,i} \phi(\|\mathbf{u} - \xi_i\|), \quad (20)$$

$$\phi(\|\mathbf{u} - \xi_i\|) = e^{-\beta \|\mathbf{u} - \xi_i\|^2}. \quad (21)$$

The activation function ϕ depends only on the distance of the input vector from a center vector, thus being radially symmetric about that vector, hence the name *radial basis function*. Depending on the RBF net model, the radial function may change as well as the metric used to estimate the distance of input from the centers. Commonly, the radial function is the Gaussian function (see Eq. (21)) while the distance metric is the Euclidian distance.

RBF networks are trained in supervised mode in a two-step training process on a set of pairs $\mathbf{v}_j, v_j$ where $\mathbf{v}_j$ is the input vector on the train set and v_j is the expected output value of the network for that input. In the first step, the center vectors ξ_i of the RBF functions in the hidden layer are chosen. Centers can be randomly sampled from some set of training inputs, or they can be determined using k -means clustering. In the second step coefficients $w_{c,i}$ of a linear model are calculated to fit the hidden layer's outputs with respect to some objective function, commonly the least squares function. A third backpropagation step can be performed to fine-tune all of the RBF net's parameters. Note that this step is unsupervised. A third backpropagation step can be performed to fine-tune all of the RBF net's parameters [28].

3.4. Single-layer lookup perceptron (SLLUP)

SLLUP [27] uses the same n -tuple sampling techniques of WiSARD though it implements arbitrary mapping functions between input vector components and RAMs in the same way as a conventional neural net.

SLLUP can be implemented using the same hardware as the WiSARD with the exceptions that each RAM cell must be capable of storing numeric values rather than just one bit. RAMs in SLLUP are partitioned into N blocks, which can be considered the homologues of discriminators in WiSARD: each block of RAMs forms a component of the output vector, which is an interpolation of a function of which we have a sparse training set of samples.

Similarly to WiSARD, the response of each block to the input pattern $\mathbf{u}$ is the linear combination of contents stored in N memory cells (one for each block) with address α defined by the n -tuple extracted from $\mathbf{u}$ by a random mapping:

$$r_c(\mathbf{u}) = \sum_{i=1}^N m_{c,i,\alpha_i(\mathbf{u})} = \sum_{i=1}^N \sum_{\alpha=0}^{2^n-1} m_{c,i,\alpha} \delta_{\alpha, \alpha_i(\mathbf{u})}. \quad (22)$$

The main difference between the two models is how the contents of memories $m_{c,i,\alpha}$ are computed during training: an n -tuple address generated by the training sample $\mathbf{v}$ is used to write corresponding memory cells in all blocks, while in WiSARD only the interested discriminator (the one devoted to the pattern class) is updated. Moreover, in SLLUP the memory contents $m_{c,i,\alpha}$ store numeric values as weights calculated during an adaptive learning procedure with a repeated application on training vectors. The adaptation of the RAM contents is performed in supervised mode by using the least mean squares algorithm to fit the actual outputs on training samples to the expected outputs.

4. The WiSARD Classifier machine learner

We propose *WiSARD Classifier* as a general-purpose classification method in ML domains. For general-purpose here we mean that the method exploits the same binarization scheme for multivariable data in the domain of real numbers, regardless on the number of data features (components) and their meaning in the specific classification problem.

The core of the method is the WiSARD model. More precisely our method relies on the DRASiW extension of the model, which has been described in Section 2.5. The reason why we adopted DRASiW is the possibility to exploit the *Bleaching algorithm* [21] to resolve classification ties and then to improve system performance (see Section 4.2).

In the following subsections we discuss: (1) the data encoding procedure we propose to transform multivariable data samples into binary inputs suitable to be processed by WiSARD, and (2) the Bleaching algorithm we use to resolve ties in classification in order to improve the system detection performance.

4.1. Data binary encoding

Although WiSARD was originally conceived as a pattern recognition device mainly focusing on image processing domain, with *ad hoc* data transformation WiSARD can also be used successfully as multiclass classifier in ML domains. We consider domains in which each datum is represented by a *feature vector*, which is a vector of real numbers. In such multivariable data domains we can adopt the well-known LibSVM [29] format to represent numeric data such that each datum (sample) of a training set can be represented in the form:

$$\text{sample} = \langle c, f_1 : v_1, \dots, f_j : v_j, \dots, f_M : v_M \rangle;$$

where c is the class identifier (a string or a number) the datum belongs to, f_j and v_j are respectively a feature identifier (a string or a number), and its value (a real number) inside the feature vector of size M representing the datum.

Before the data could be processed by the WiSARD, they have to be converted to binary patterns. First of all, feature values v_j in the range $[\alpha_j, \beta_j]$ have to be discretized and scaled to the corresponding integer values $\underline{v}_j$ in the interval $[0, z-1]$. With this transformation any real number $v_j \in [\alpha_j, \beta_j]$ will be represented by the non-negative integer:

$$\underline{v}_j = \left\lceil \frac{(v_j - \alpha_j) \times z}{\beta_j - \alpha_j} \right\rceil. \quad (23)$$

Thus, under the transformation of Eq. (23), the format of dataset samples becomes:

$$\text{sample} = \langle c_i, f_1 : \underline{v}_1, \dots, f_j : \underline{v}_j, \dots, f_M : \underline{v}_M \rangle;$$

where the feature vector has as elements all non-negative integers in the range $[0, z-1]$. For example, let us consider a training set of class c , called TS_c , composed of 4 samples ($|TS_c| = 4$), with each data sample having $M = 3$ feature values. Suppose that, after scaling and discretization of feature values to integers in the range $[0, 4]$ ($z = 5$), TS_c will contain the following samples:

$$\begin{aligned} \text{sample}_1 &= \langle 1, f_1 : 2, f_2 : 1, f_3 : 3 \rangle \\ \text{sample}_2 &= \langle 1, f_1 : 2, f_2 : 2, f_3 : 3 \rangle \\ \text{sample}_3 &= \langle 1, f_1 : 3, f_2 : 2, f_3 : 4 \rangle \\ \text{sample}_4 &= \langle 1, f_1 : 4, f_2 : 0, f_3 : 4 \rangle. \end{aligned} \quad (24)$$

Hence, each data sample can be represented by a $M \times (z-1)$ binary pattern adopting the *thermometer code* [30], that is a binary

matrix having as column j a sequence of $z-1$ bits, where the first consecutive $\underline{v}_j$ bits are set to 1. The thermometer code guarantees that close values of $\underline{v}_j$ correspond to binary patterns with small Hamming distance. As an example, the samples of Eq. (24) are transformed by the thermometer code into the binary patterns of Fig. 2.

With the transformation of Eq. (23), numeric datasets can be used both for training and for classification in WiSARD systems. With other *ad hoc* transformations, WiSARD can even deal with non-numeric data, like nominal data⁵ and ordinal datatypes in symbolic domains [10].

4.2. Resolving ties

Bleaching is an algorithm developed by [21] that exploits the capability of DRASiW networks (see Section 2.5) to store frequency information in RAMs to eliminate discriminators' response ties in the following way:

1. a *step variable* $b \geq 1$ and a starting threshold $\tau = 0$ for bleaching are set;
2. upon classification of a new sample, all RAMs, whose accessed cells have content (n -tuple frequency) greater than or equal to τ , outputs 1, while the rest outputs 0;
3. RAM outputs are collected as in the original model to form the discriminators' responses (see Eq. (6));
4. if there are ties among discriminators' responses, increase τ by b and repeat step 2;
5. If there is no tie, the discriminator with the highest response classifies the input sample.

Ties in discriminators' responses are detected when discriminators' outputs are close within a *tie confidence* tolerance, named c_b . As a result of bleaching, sub-patterns that the network considered too uncommon (i.e., the ones that occurred less than τ times during training) do not contribute to discriminators' responses.

Although the Bleaching algorithm was designed to resolve classification ties to improve overall detection, it has also the advantage of mitigating effects of outliers that might be present in the training sets.

To clarify this point, let us consider the case where one sample (say s) belonging to a certain class C_1 is also used in the training of class C_2 . This eventuality could compromise a detection performance of a system based on the pure WiSARD model: indeed, from Eq. (13), it is clear how an input sample close (in terms of Hamming distance) to s will bring the classifier to respond either C_1 or C_2 with very close similarity scores.

This effect can be quite muted and absorbed in our approach since it relies on the DRASiW extension: RAM contents corresponding to n -tuples of outliers will store frequency values close to zero, while the memory contents corresponding to other n -tuples (produced by proper samples of the training set) will store higher frequency values. The bleaching strategy will prevent to take into account less-frequent n -tuples (generated by outliers) since they will be cut off by the bleaching thresholding.

5. Experiments

We evaluated the performance of *WiSARD Classifier* by examining its behavior on the *standard classification datasets* available on the KEEL Archive.⁶ From this set of 76 classification problems we ignored 6 datasets due to the very time-consuming training

⁵ Also called *categorical*.

⁶ <http://sci2s.ugr.es/keel>.

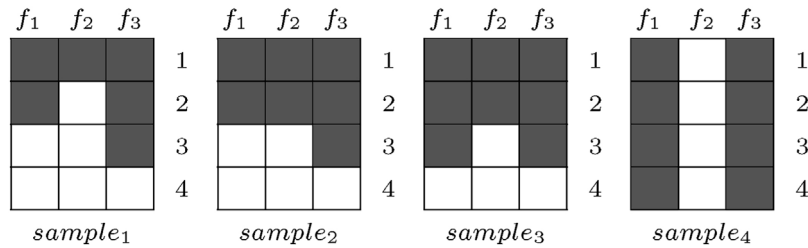


Fig. 2. Binary patterns representing samples of Eq. (24).

Table 1
Methods and hyper-parameters' configurations used for comparative analysis. Bold values are ranges of parameter values on which model selection process is carried out.

Acronym	Name	Hyper-parameters' configuration
<i>Base learners</i>		
WNN	Wisard Classifier	(<i>n</i>) bit addressing = [4,8,16,32,64], (<i>z</i>) data scaling factor = [64,128,256,512,1024,2048,4096,8192], (<i>b</i>) step variable of bleaching = 1, (<i>c_b</i>) tie confidence = 0.01
MLP	Multi Layer Perceptron	solver for weight optimization = [l-BFGS,ADAM], one hidden layer with size = [100,200,330], RELU activation function, L2 penalty regularization term = 10^{-4} , constant learning rate = 10^{-3}
SVC	Support Vector Machine	kernel function = [linear, rbf], penalty parameter C = [0.001,0.01,0.1,1], kernel coefficient for rbf gamma = [0.001,0.01,0.1,1], tolerance for stopping criterion = 10^{-3} ;
LR	Logistic Regression	Inverse of regularization strength C = [0.001,0.01, 0.1,1], penalization norm = [L1,L2], solver = Liblinear, Tolerance for stopping criteria = 10^{-4}
GNB	Gaussian Naive Bayes	none
kNN	k-Nearest Neighbors	algorithm = [BallTree,KDTree,brute force], no. of neighbors = [3,5,10,15,30], euclidean distance, uniform weights for neighborhood points, leaf-size = 30
<i>Ensemble learners</i>		
GTB	Gradient Tree Boosting	no. of estimators = [100,200,300], max depth of tree = [2,3,5,10], unlimited maximum number of terminal nodes or leaves in a tree (bounded to 2^d leaves where <i>d</i> is maximum depth), learning rate = 0.1, split quality measure = Friedman mean squared error, no. of features for best split = no. of classes
RF	Random Forrest	no. of trees in forest = [100,200,300], no limit on tree dept, split quality measure = [gini, entropy], no. of features for best split = $\sqrt{\text{no. of features}}$
AB	AdaBoost	no. of estimators = [20,50,100], algorithm = [SAMME,SAMME.R]

times. Nonetheless, in our experimental study we experimented 8 ML methods, namely *benchmarking methods*. They competed with our *WiSARD Classifier* on 70 classification problems coming from very different domains. The experimental study was large enough to have an exhaustive statistical analysis of results that allowed us to assert general statements on performance of the methods under study.

Table 1 lists the benchmarking methods we have compared to *WiSARD Classifier*. The methods' implementation is available in Scikit Learn library.⁷ The table is divided into two parts: the top part lists the *base learners*, i.e. classification methods implementing one specific learning algorithm; the bottom part reports the *ensemble learners* [31], i.e. methods that use multiple learning algorithms to obtain better predictive performance than could be obtained from any of the constituent learning algorithm by alone. Among the *base learners* we selected: a Multi-Layer Perceptron (MLP), a Support Vector Machine (SVC), a linear-model method based on Logistic Regression (LR), and a *k*-Nearest Neighbors (*k*NN) clustering method. In the set of *ensemble learners* all methods fall in the category of decision-tree learning algorithms, although differing in the way ensembles are built up: Adaboost (AB), which is a boosting ensemble of decision-trees exploiting the SAMME algorithm [31], Gradient Tree Boosting [32] (GTB), which exploits boosting methods to combine decisions-tree classifiers, and Random Forest (RF), which is a bagging meta-estimator based on decisions-tree classifiers using the Gini Index [33] criterion to evaluate split quality.

The aim of the experimental phase is to compare accuracy of *WiSARD Classifier* with that of other classification methods available in the Scikit Learn library (listed in Table 1). To pursue this goal

we first carried out model selection process for the choice of hyper-parameters by exhaustively search in a parameter grid to generate candidates by means of a five-fold cross-validation.⁸ Best parameters were evaluated for each method in relation to each dataset (they are reported in Tables 10 and 11). In Table 1 bold parameters are those we considered for the tuning in model selection, while the rest of parameters were fixed to the defaults.

The meaning of *WiSARD Classifier* parameters and the ranges of values searched in the model selection process are hereafter explained:

- *n* = bit addressing, i.e. the *n*-tuple size in *WiSARD* (tuned by values: 2^k with $k \in [1, \dots, 6]$)
- *z* = data scaling factor, where $[0, z - 1]$ is the integer range to which each real value of the multivariable datum is discretized before binary encoding (tuned by values: 2^k with $k \in [6, \dots, 13]$)
- *b* = step variable of the Bleaching algorithm (fixed to 1);
- *c_b* = tie confidence tolerance of the Bleaching algorithm (fixed to 0.01).

In all experiments the Bleaching algorithm was always enabled.

Once the best parameters were found, we measured the average of classification accuracies of each method in a ten-fold stratified⁹ cross-validation on each dataset. This means that each dataset was randomly partitioned into ten equal sized subsamples (balanced due to stratification modality). Then, for each method we performed ten rounds of validations: in each run, a different subsample of the ten was retained as validation data for testing, and

⁸ By means of the GridSearchCV utility of Scikit Learn library.

⁹ i.e. the folds are selected so that the mean response value is approximately equal in all the folds.

⁷ <http://scikit-learn.org>.

Table 2

Average accuracy of methods over a ten-fold stratified cross-validation on datasets.

Dataset (size: features)	Methods								
	LR	GNB	kNN	SVC	RF	AB	CTB	MLP	WNN
Abalone (4174:10)	0.2555	<u>0.0984</u>	0.2510	0.2603	0.2533	0.2105	0.2672	0.2706	0.2702
adult (48842:104)	0.8523	<u>0.6067</u>	0.8361	0.8511	0.8545	0.8655	0.8746	0.8517	0.8301
appendicitis (106:7)	<u>0.8023</u>	0.8479	0.8379	0.8779	0.8679	0.8480	0.8703	0.8670	0.8585
australian (690:14)	0.8565	<u>0.7969</u>	0.8566	0.8609	0.8608	0.8566	0.8666	0.8566	0.8623
automobile (159:71)	0.6741	<u>0.5722</u>	0.6407	0.7471	0.8809	0.6122	0.8705	0.7714	0.8176
balance (625:4)	0.8607	<u>0.8381</u>	0.8369	0.8881	<u>0.7120</u>	0.8417	0.8083	0.9729	0.7808
banana (5300:2)	<u>0.5709</u>	0.6140	0.9036	0.6504	0.8970	0.7168	0.8989	0.8949	0.9030
bands (365:19)	0.6274	<u>0.3729</u>	0.5755	0.6276	0.5806	0.6058	0.5730	0.6025	0.6493
breast (263:9)	0.7300	<u>0.7263</u>	<u>0.6730</u>	0.7150	0.6772	0.7066	0.6837	0.7228	0.7072
bupa (345:6)	0.6463	<u>0.5622</u>	0.5911	0.5943	0.7363	0.7361	0.7334	0.7448	0.6781
car (1728:21)	0.8209	<u>0.8029</u>	0.7783	0.8613	0.8464	0.8033	0.8960	0.8881	<u>0.7772</u>
chess-2 (3196:38)	0.9424	<u>0.6097</u>	0.7935	0.9114	0.9612	0.9396	0.9812	0.9796	0.9143
cleveland (297:13)	0.5892	<u>0.5345</u>	0.5534	0.5656	0.5697	0.5623	0.5426	<u>0.5023</u>	0.5892
coil2000 (9822:85)	0.9402	<u>0.1355</u>	0.9405	0.9403	0.9290	0.9403	0.9394	0.9371	0.9403
connect-41 (67557:126)	0.7259	<u>0.4511</u>	0.6155	0.6478	0.6052	0.7174	0.6197	0.6506	0.6641
contraceptive (1473:9)	0.5092	<u>0.4664</u>	<u>0.4480</u>	0.5282	0.5193	0.5458	0.5669	0.5547	0.5255
crx (653:42)	0.8620	<u>0.6848</u>	0.8360	0.8636	0.8603	0.8484	0.8421	0.8466	0.8391
dermatology (358:34)	0.9743	<u>0.8656</u>	0.9629	0.9716	0.9776	<u>0.5363</u>	0.9526	0.9662	0.9832
e. coli (336:7)	0.7658	<u>0.5902</u>	0.7945	0.8288	0.8544	0.7816	0.8308	0.8119	0.8482
flare (1066:38)	0.7422	<u>0.6135</u>	0.7402	0.7365	0.7180	0.7020	0.7461	0.7083	0.7495
german (1000:24)	0.7580	<u>0.7300</u>	<u>0.7190</u>	0.7660	0.7680	0.7450	0.7680	0.7520	0.7460
glass (214:9)	0.5802	<u>0.4690</u>	0.6695	0.5906	0.7926	0.5146	0.7249	0.6893	0.7243
haberman (306:3)	0.7485	<u>0.7424</u>	0.7354	0.7353	<u>0.6862</u>	0.7355	0.7133	0.7389	0.7320
hayes-roth (160:4)	0.5106	0.6983	<u>0.3754</u>	0.6735	0.8266	0.6001	0.8356	0.8045	0.8000
heart (270:13)	0.8444	0.8407	0.8259	<u>0.8111</u>	0.8222	0.8259	0.8111	0.8444	0.8333
hepatitis (80:19)	0.8405	<u>0.5919</u>	0.8181	0.8405	0.8891	0.8827	0.8323	0.8038	0.8750
housevotes (232:16)	0.9610	<u>0.9479</u>	<u>0.9184</u>	0.9693	0.9608	0.9608	0.9693	0.9569	0.9569
ionosphere (351:33)	0.8641	0.8866	<u>0.8469</u>	0.9408	0.9297	0.9128	0.9211	0.9180	0.9487
iris (150:4)	<u>0.9000</u>	0.9533	0.9533	0.9667	0.9600	0.9467	0.9600	0.9600	0.9467
led7digit (500:7)	0.7403	0.6957	0.7417	0.7478	0.7111	<u>0.6867</u>	0.7312	0.7543	0.7220
letter (20000:16)	0.7234	<u>0.6424</u>	0.9562	0.9042	0.9679	<u>0.4204</u>	0.9637	0.8581	0.935
lymp (148:38)	0.8585	<u>0.5839</u>	0.7744	0.8685	0.8313	<u>0.7307</u>	0.8975	0.8498	0.8514
magic (19020:10)	0.7905	<u>0.7269</u>	0.8392	0.8562	0.8815	0.8460	0.8756	0.8613	0.8489
mammographic (830:5)	0.8073	<u>0.8060</u>	0.8023	0.7929	<u>0.7866</u>	0.8288	0.8299	0.8035	0.8096
marketing (6876:13)	0.3172	<u>0.3068</u>	<u>0.3068</u>	0.3374	0.3171	0.3239	0.3433	0.3350	0.3394
monk-2 (432:6)	<u>0.7665</u>	0.8753	0.9424	0.9721	0.9814	1.0000	1.0000	0.9977	0.9815
movement-libras (360:90)	0.6667	0.6033	0.8067	0.8178	0.8211	<u>0.3211</u>	0.7056	0.8533	0.8250
mushrooms (8124:112)	0.9967	0.9605	0.9631	1.0000	0.9678	0.9656	0.9998	0.9824	0.9685
new thyroid (215:5)	<u>0.8885</u>	0.9677	0.9437	0.9208	0.9582	0.9537	0.9487	0.9626	0.9721
nursery (12960:26)	0.8521	<u>0.8007</u>	0.8083	0.9788	0.8974	0.8089	0.9983	0.9926	0.8600
optdigits (5620:64)	0.9583	<u>0.7749</u>	0.9831	0.9843	0.9803	0.8483	0.9749	0.9799	0.9754
page-blocks (5472:10)	0.9445	<u>0.8864</u>	0.9519	0.9361	0.9686	0.9541	0.9698	0.9636	0.9567
penbased (10992:16)	0.9306	0.8559	0.9931	0.9953	0.9915	<u>0.6912</u>	0.9915	0.9922	0.9916
phomene (5404:5)	0.7496	0.7605	0.8901	0.7970	0.9130	0.8231	0.9099	0.8458	0.8990
pima (768:8)	0.7707	<u>0.7629</u>	0.7590	0.7837	<u>0.7524</u>	0.7564	0.7564	0.7746	0.7760
post-operative (87:21)	0.7170	<u>0.3635</u>	0.7045	0.7170	<u>0.6593</u>	0.6454	0.6374	0.6138	0.6897
ring (7400:20)	0.7613	0.9796	<u>0.7207</u>	0.9786	0.9515	0.9695	0.9682	0.9509	0.7292
saheart (462:9)	0.7273	0.7100	<u>0.7077</u>	0.7338	0.6819	0.6859	<u>0.6408</u>	0.7078	0.7294
satimage (6435:36)	0.8236	0.7932	0.8925	0.8909	0.9040	<u>0.7748</u>	0.9001	0.8814	0.8920
segment (2310:19)	0.9108	<u>0.7987</u>	0.9610	0.9437	0.9784	0.8065	0.9818	0.9662	0.9714
shuttle (57999:9)	0.9280	<u>0.8057</u>	0.9991	0.9858	0.9999	0.9889	0.9998	0.9858	0.9977
sonar (208:60)	0.6604	<u>0.6036</u>	0.6432	0.6441	0.7328	0.6985	0.7092	0.6952	0.6971
spambase (4597:57)	0.9119	<u>0.8211</u>	0.8878	0.9230	0.9458	0.9391	0.9454	0.9365	0.9028
spectfheart (267:44)	0.7943	<u>0.6858</u>	0.7792	0.7943	0.8273	0.7906	0.7973	0.7943	0.7978
splice (3175:60)	0.8466	0.8709	<u>0.7836</u>	0.9121	0.9701	0.9398	0.9717	0.9235	0.9389
tae (151:5)	0.4837	0.5163	0.4935	<u>0.3440</u>	0.7029	0.4758	0.6838	0.5979	0.6887
texture (5500:40)	0.9818	0.7744	0.9876	0.9927	0.9796	<u>0.6331</u>	0.9864	0.9956	0.9769
thyroid (7200:21)	0.9455	<u>0.1235</u>	0.9404	0.9385	0.9956	0.9894	0.9962	0.9843	0.9425
tic-tac-toe (958:27)	0.9832	<u>0.6267</u>	0.8751	0.9832	0.9251	0.8780	0.9729	0.9603	0.9207
titanic (2201:3)	0.7760	<u>0.7733</u>	<u>0.7287</u>	0.7819	0.7878	0.7783	0.7905	0.7892	0.7878
twonorm (7400:20)	0.9778	0.9786	0.9749	0.9785	0.9736	<u>0.9673</u>	0.9734	0.9773	0.9759
vehicle (846:18)	0.7533	<u>0.4611</u>	0.6915	0.7567	0.7483	0.6089	0.7745	0.8181	0.7435
vowel (990:13)	0.4404	0.5970	0.6232	0.7040	0.7364	0.3505	0.6343	0.7495	0.7364
wdbc (569:30)	0.9649	<u>0.9315</u>	0.9702	0.9772	0.9650	0.9614	0.9667	0.9649	0.9684
wine (178:13)	0.9666	<u>0.9663</u>	0.9712	0.9826	0.9774	<u>0.9215</u>	0.9329	0.9826	0.9888
winequality-red (1599:11)	0.5705	0.5368	<u>0.5216</u>	0.5786	0.5736	0.5336	0.5329	0.5962	0.5822
winequality-white (4898:11)	0.5289	<u>0.4342</u>	0.4962	0.5199	0.5366	0.4751	0.5140	0.5366	0.5439
wisconsin (683:9)	0.9693	<u>0.9620</u>	0.9723	0.9693	0.9650	<u>0.9562</u>	0.9634	0.9650	0.9722
yeast (1484:8)	0.5773	<u>0.1366</u>	0.5730	0.5892	0.6021	0.4931	0.5900	0.5879	0.5943
zoo (101:21)	<u>0.9053</u>	0.9624	0.9701	0.9638	0.9840	0.9141	0.9826	0.9638	0.9703

Table 3

Methods' ranking according to three nonparametric tests: R_f = Friedman test ranking, R_{fa} = Friedman Aligned Rank test ranking, R_q = Quade test ranking.

Algorithm	Statistica tests		
	R_f	R_{fa}	R_q
GTB	3.74	231.64	3.54
WNN	3.92	240.09	3.94
RF	3.96	232.51	3.51
MLP	4.03	225.64	4.03
SVC	4.11	260.19	4.31
LR	5.41	362.55	5.58
AB	6.15	401.89	6.16
kNN	6.18	371.51	6.00
GNB	7.50	513.48	7.93
Statistics	22.36	176.18	20.50
p -value	1.11e-16	0.00e+00	1.11e-16

the remaining nine subsamples were used as training. The final accuracy measure for each method/dataset pair is the average of the accuracies obtained in the ten runs. Of course, the random partitioning of each dataset was the same for all methods. In all experiments no features selection was carried out. Before applying methods to each dataset¹⁰ a data preprocessing step is performed to minmax scaling¹¹ of feature values to the range [0, 1] of real numbers.

The average accuracy measured for each method on each dataset is reported in Table 2. We highlighted in bold the best average accuracy across all methods, while in underlined text the worst performances are marked.

6. Methods comparison through nonparametric statistical analysis

We carried out a statistical analysis of experimental results by using a classical approach based on multiple nonparametric statistical tests [34,35]. Our hypothesis is the following:

H_0 : The accuracy distributions over N datasets of methods are the same;

where $N = 70$. We chose three nonparametric statistical tests, namely the Friedman test, Friedman Aligned Rank test and Quade test, to evaluate the statistical error when rejecting (or accepting) the null-hypothesis H_0 .

These tests were chosen based on the assumptions that average accuracy data are not distributed according to a normal distribution and that the number of algorithms to compare is $k = 9$.

The three tests define different rankings of methods' performance. As showed in Appendix A, both Friedman and Friedman Aligned Rank tests use χ^2 distribution of rankings, although their formulas to get a χ^2 distribution differ, while the Quade test uses a F distribution of rankings and it uses a different formula to compute a χ^2 distribution based on the rankings.

In all tests we assume a significant level (threshold) of $\alpha = 0.05$, that is whether we get from the statistic a p -value $< \alpha$ we reject the null hypothesis, otherwise it is significantly true.

6.1. Statistical analysis

In Table 3 we report the methods' ranking according to the three statistic tests. The table is ordered bottom-up according to the Friedman test ranking (R_f). It is to notice how *WiSARD Classifier*

scored 2nd in the Friedman ranking, 4th in the Friedman Aligned ranking and 3rd in the Quade ranking.

Friedman test statistic is 22.36 (distributed according to χ^2 with 8 degrees of freedom), and the corresponding p -value is 1.11×10^{-16} . Since p -value $< \alpha$, then:

the null-hypothesis H_0 is rejected by the Friedman test.

Friedman Aligned Rank test statistic is 176.18 (distributed according to χ^2 with 8 degrees of freedom), and the corresponding p -value is 0.00. Since p -value $< \alpha$, then:

the null-hypothesis H_0 is rejected by the Friedman Aligned Rank test.

Quade test statistic is 20.50 (distributed according to F -distribution with 8 and 552 degrees of freedom), and the corresponding p -value is 1.11×10^{-16} . Since p -value $< \alpha$, then:

the null-hypothesis H_0 is rejected by the Quade test.

Therefore, within the chosen statistical significant level α (0.05) for all tests we reject the H_0 hypothesis, that is we can assume the k methods did not perform in the same ways on the N datasets. Of course, this may hold if at least two methods in the set of classifiers differ in their performance.

In order to have a more accurate statistical analysis of experimental results, in the following section we carry out post-hoc statistical procedures to compare pairwise each method performance with respect to a control method, which is, respectively, *GTB* for the Friedman test, *MLP* for the Friedman Aligned Rank test, and *RF* for the Quade test.

In Table 4 the Contrast Estimation matrix of the methods is presented. As one can notice, the row corresponding to method *GTB* has all positive values, since it is the best performing method.

7. Multiple comparisons with control methods

Since in the previous section we have rejected the null hypothesis in all tests, and in each test a different best performing method was found, we can now focus on the comparison between the best performing method, namely the *control method*, and the other methods. For each test we carried out $1 \times k$ comparisons associated with a set (or family) of hypotheses, all of which are related to the reference control method. In particular we consider the following sets of hypotheses, where each set H_X^t of hypotheses refers to a different control method computed by the three tests: Friedman ($t = f$), Friedman Aligned Rank ($t = fa$), and Quade test ($t = q$):

- H_X^f : The accuracy distribution over N datasets for method X is the same of that of method *GTB*;
- H_X^{fa} : The accuracy distribution over N datasets for method X is the same of that of method *MLP*;
- H_X^q : The accuracy distribution over N datasets for method X is the same of that of method *RF*;

where X is any method (listed in Table 7) different from the control method.

Hypotheses testing has been carried out with Bonferroni-Dunn, Holm, Hochberg, Finner and Li post-hoc procedures for $1 \times k$ comparisons, where control method is *GTB*, *MLP* and *RF* respectively in the case of Friedman (see Table 5), Friedman Aligned Rank (see Table 6), and Quade test (see Table 7). In the tables, methods are listed in columns from left to right following the decreasing of p -values adjusted by the Bonferroni-Dunn procedure. In all tests Bonferroni-Dunn procedure provides the highest adjusted p -values, whereas Finner and Li procedure give the lowest adjusted p -values. The Bonferroni-Dunn procedure is the simplest procedure

¹⁰ We used the Pipeline utility of Scikit Learn.

¹¹ By mean of the MinMaxScaler utility of Scikit Learn.

Table 4

Matrix of contrast estimation between medians of samples in pairwise comparison of methods.

	LR	GNB	kNN	SVC	RF	AB	GTB	MLP	WNN
LR	0.00e+00	6.95e−02	3.96e−03	−1.15e−02	−1.81e−02	8.30e−03	−1.93e−02	−1.73e−02	−1.69e−02
GNB	−6.95e−02	0.00e+00	−6.55e−02	−8.10e−02	−8.76e−02	−6.12e−02	−8.88e−02	−8.69e−02	−8.64e−02
kNN	−3.96e−03	6.55e−02	0.00e+00	−1.54e−02	−2.21e−02	4.34e−03	−2.32e−02	−2.13e−02	−2.09e−02
SVC	1.15e−02	8.10e−02	1.54e−02	0.00e+00	−6.62e−03	1.98e−02	−7.77e−03	−5.86e−03	−5.44e−03
RF	1.81e−02	8.76e−02	2.21e−02	6.62e−03	0.00e+00	2.64e−02	−1.15e−03	7.61e−04	1.17e−03
AB	−8.30e−03	6.12e−02	−4.34e−03	−1.98e−02	−2.64e−02	0.00e+00	−2.76e−02	−2.56e−02	−2.52e−02
GTB	1.93e−02	8.88e−02	2.32e−02	7.77e−03	1.15e−03	2.76e−02	0.00e+00	1.91e−03	2.32e−03
MLP	1.73e−02	8.69e−02	2.13e−02	5.86e−03	−7.61e−04	2.56e−02	−1.91e−03	0.00e+00	4.11e−04
WNN	1.69e−02	8.64e−02	2.09e−02	5.44e−03	−1.17e−03	2.52e−02	−2.32e−03	−4.11e−04	0.00e+00

Table 5*p*-Values of Friedman test adjusted by Bonferroni-Dunn, Holm, Hochberg, Finner and Li procedure.

<i>p</i> -Value	Pairwise comparison of methods GTB vs							
	WNN	RF	MLP	SVC	LR	AB	kNN	GNB
z-value	3.86e−01	4.78e−01	6.17e−01	7.87e−01	3.60e+00	5.20e+00	5.26e+00	8.12e+00
Unadjusted <i>p</i>	7.00e−01	6.32e−01	5.37e−01	4.31e−01	3.24e−04	1.99e−07	1.43e−07	4.44e−16
<i>p</i> _{Bonf}	1.00e+00	1.00e+00	1.00e+00	1.00e+00	2.59e−03	1.59e−06	1.14e−06	3.55e−15
<i>p</i> _{Holm}	1.00e+00	1.00e+00	1.00e+00	1.00e+00	1.62e−03	1.20e−06	9.99e−07	3.55e−15
<i>p</i> _{Hoch}	7.00e−01	1.00e+00	1.00e+00	1.00e+00	1.00e+00	1.00e+00	1.00e+00	1.00e+00
<i>p</i> _{Finn}	7.00e−01	6.81e−01	6.42e−01	5.95e−01	6.48e−04	5.71e−07	5.71e−07	3.55e−15
<i>p</i> _{Li}	7.00e−01	6.78e−01	6.41e−01	5.90e−01	1.08e−03	6.63e−07	4.75e−07	1.48e−15

Table 6*p*-values of Friedman Aligned Rank test by Bonferroni-Dunn, Holm, Hochberg procedure.

<i>p</i> -Value	Pairwise comparison of methods MLP vs							
	GTB	RF	WNN	SVC	LR	kNN	AB	GNB
z-value	1.95e−01	2.23e−01	4.70e−01	1.12e+00	4.45e+00	4.74e+00	5.73e+00	9.36e+00
Unadjusted <i>p</i>	8.46e−01	8.23e−01	6.39e−01	2.61e−01	8.58e−06	2.12e−06	1.01e−08	0.00e+00
<i>p</i> _{Bonf}	1.00e+00	1.00e+00	1.00e+00	1.00e+00	6.87e−05	1.70e−05	8.10e−08	0.00e+00
<i>p</i> _{Holm}	1.00e+00	1.00e+00	1.00e+00	1.00e+00	4.29e−05	1.27e−05	7.09e−08	0.00e+00
<i>p</i> _{Hoch}	8.46e−01	1.00e+00	1.00e+00	1.00e+00	1.00e+00	1.00e+00	1.00e+00	1.00e+00
<i>p</i> _{Finn}	8.62e−01	8.62e−01	7.43e−01	3.84e−01	1.72e−05	5.66e−06	4.05e−08	0.00e+00
<i>p</i> _{Li}	8.46e−01	8.42e−01	8.05e−01	6.29e−01	5.56e−05	1.37e−05	6.56e−08	0.00e+00

Table 7*p*-values of Quade test adjusted by Bonferroni-Dunn, Holm, Hochberg, Finner and Li procedure.

<i>p</i> -Value	Pairwise comparison of methods RF vs							
	GTB	WNN	MLP	SVC	LR	kNN	AB	GNB
z-value	2.83e−02	3.96e−01	4.86e−01	7.45e−01	1.94e+00	2.33e+00	2.48e+00	4.15e+00
unadjusted <i>p</i>	9.77e−01	6.92e−01	6.27e−01	4.56e−01	5.28e−02	1.97e−02	1.30e−02	3.40e−05
<i>p</i> _{Bonf}	1.00e+00	1.00e+00	1.00e+00	1.00e+00	4.22e−01	1.58e−01	1.04e−01	2.72e−04
<i>p</i> _{Holm}	1.00e+00	1.00e+00	1.00e+00	1.00e+00	2.64e−01	1.18e−01	9.10e−02	2.72e−04
<i>p</i> _{Hoch}	9.77e−01	1.00e+00	1.00e+00	1.00e+00	1.00e+00	1.00e+00	1.00e+00	1.00e+00
<i>p</i> _{Finn}	9.77e−01	7.40e−01	7.31e−01	6.23e−01	1.03e−01	5.18e−02	5.10e−02	2.72e−04
<i>p</i> _{Li}	9.77e−01	9.68e−01	9.65e−01	9.53e−01	7.00e−01	4.66e−01	3.65e−01	1.50e−03

but it is also the least powerful, while the Li and Finner procedure seem to be the multiple comparison tests that exhibit the most powerful behavior reaching the lowest *p*-values.

In all tables we indicated with italic numbers the cases in which hypotheses H_X^t are rejected, that is the hypotheses for which all adjusted *p*-values are less or equal to the significance level α .

In the Friedman test (see Table 7), with regards to Bonferroni-Dunn, Holm, Finner, and Li procedure, accepted hypotheses H_X^f are those comparing the control method GTB with RF, MLP, SVC, and WNN, while Hochberg procedure accepts all hypotheses.

In the Friedman Aligned Rank test (see Table 7), with regards to Bonferroni-Dunn, Holm, Finner, and Li procedure, accepted hypotheses H_X^{fa} are those comparing the control method MLP with

GTB, RF, SVC, and WNN, while Hochberg procedure accepts all hypotheses.

Since the Finner and Li procedure are the most powerful, we consider the set of methods whose hypotheses are accepted by these two procedures in both the Friedman and Friedman Aligned tests. With this choice, methods GTB, MLP, RF, SVC and WNN are equivalent in terms of performance, although the closer is the *p*-value to 1, the higher is the level of acceptance. In this extension, we can say that, in the case of Friedman test and with respect to the *p*-values adjusted by the Li procedure, our method WNN is the closest to the control method GTB with a 70% level of acceptance, while the other equivalent methods (i.e. RF, MLP and SVC) have lower acceptance levels ranging from 60% to 68%. By analyzing *p*-values in the case of the Friedman Aligned Rank test which are adjusted by the Finner

and Li procedure, levels of acceptance are higher. With respect to p -values adjusted by the Li procedure, *GTB* and *RF* are the closest to the control method *MLP* with about 85% of acceptance level, while our method shows an 80% of acceptance level which is higher than that of *SVC*.

Post-hoc procedures for the Quade test confirm the same results of the previous tests regarding the equivalence of methods *GTB*, *MLP*, *SVC*, and *WNN* with respect to the control method *RF*. In addition, very low levels of acceptance are granted for the equivalence of methods *LR*, *kNN* and *AB* to *RF*. Reminding the ranking scheme of the Quade test (see Appendix A), this result supports the conclusion that, although *RF* obtains a better ranking than the others (including our method *WNN*), these methods are equivalent in the sense that they behave similarly in the most difficult problems, and thus performance differences are not detected if the relative difficulties of the problems are taken into account.

8. Multiple comparisons among all methods

Friedman test, as well as Friedman Aligned Rank test and Quade test, are omnibus tests which have been used in this work to detect differences in classification accuracy considering the global set of classification methods. Once those tests all together rejected the null hypothesis, we proceeded with post-hoc tests in order to find the concrete pairwise comparisons which produce differences between each method and the control methods *GTB*, *MLP* and *RF* as computed respectively by the three tests. This post-hoc analysis served in our aims to assess that our classification method based on *WiSARD* does not produce significant differences in accuracy with respect to the control methods which behaved at best over the N datasets. We underline the fact that the p -values computed by the post-hoc procedures in the three tests confirmed that, within a significant acceptance level, our method can be considered, in terms of average accuracy obtained across 70 dataset, equivalent to the control methods.

In this section, we are interested in carrying out a multiple comparison in which all possible pairwise comparisons of methods are computed ($k \times k$ comparison). Classic procedures that can be used for the purpose are Nemenyi, Holm, Hockberg, Finner and Shaffer test, which account for the fact that all pairwise comparisons are logically interrelated, thus not all combinations of true and false hypotheses are possible. All tests adjust the p -value of each pairwise comparison of methods before checking if it is lower than the significant level α (chosen as 0.05). The results, limited to the comparisons of *WNN* with the rest of algorithms, are reported in Table 8. In this table, cells with numbers in italic represent the rejected null hypotheses that is those hypotheses for which the adjusted p -values are less or equal to the significance level α .

By considering in Table 8 as equivalent the pairs of methods for which all tests provide acceptance of the hypotheses, the following conclusions hold for the p -values adjusted by the Finner procedure: method *WNN* outperforms methods like *LR*, *AB*, *kNN*, and *GNB*. In addition, by looking at the p -value order of magnitude, which is expressive about “how significant” is the null hypothesis retention (or rejection), we can assess that *WNN* is closer in performance to methods *MLP* and *RF* with, respectively, levels of acceptance of 85% and 93% in the Finner test. This is in agreement with the small gaps between these methods which are reported in the Friedman ranking (see Table 8). In our opinion, this is a good result if we consider that *MLP* is a fully-connected three-layer neural network, while our neural model has a simpler architecture with a one-layer network of nodes each one mapping only one input portion; on the other hand, *RF*, which is well-known in literature as one of the best method, is an *ensemble method* that combines multiple decision-tree classifiers by means of bagging to obtain bet-

ter predictive performance. On the contrary, *WNN* is not based on ensemble learning.

Although methods *GTB* and *SVC* perform, respectively, better and worse than *WNN*, by looking at the p -value adjusted by the Finner procedure, we can notice that the levels of acceptance of the null hypothesis in both cases are the same (around 76%). This is in agreement with Friedman ranking (see Table 8) since the gaps between *WNN* and these methods are larger but similar.

9. Timing

We performed timing measurements of the five best methods as resulting from our experiments. We measured training and testing times of each method on a subset of the 70 datasets chosen in the previous experiments, by configuring each method with the optimal parameters found (see Tables 10 and 11) by the model selection process executed before the experiments.

The timing results are reported in Table 9. For each problem we randomly split the dataset to obtain a test set with 10% of the total samples, while the remaining samples were used for training. The same pair of train and test set has been used for all methods. Time measures are averaged over ten runs of each method on the dataset split.

All methods run on a 4 GHz Intel QuadCore i7 (iMac) with 32 GB RAM. Only two methods, specifically *WNN* and *RF*, exploit multicore hardware by means of Python `multiprocessing` library. A star symbol in the table header indicates time measures of parallel executions. In particular, since for *RF* we do not know which of the processing phases is actually parallelized,¹² we indicate in the table both training and testing times as “parallel”. On the other hand, method *WNN* exploits parallel computation only in the testing phase, whereas the set of testing samples are concurrently classified in chunks sized in such a way to have a number of chunks equal to the number of cores (4).

The first consideration is about comparing training and testing time of method *WNN*: classification time is generally in the same order or less than one order of magnitude than the training time, while most of the benchmarking methods show testing times some orders of magnitude less than training times.

The train-to-test timing ratio of our method derives from the complexity analysis of the *WiSARD* model of computation that follows. In the training phase, the set of samples is logically partitioned into classes of patterns. Apart from the encoding step of each multivariable datum (a sample) into a set of binary keys, we have that each training sample causes accesses to all N RAMs of one discriminator (associated to the class of reference). Thus, the training time goes as $O(N)$. We know that $N = \frac{z \cdot M}{n}$, where z is the thermometer code's range size, M is the number of variables in the sample (features), and n is the key (tuple) size. While z and n are hyperparameters of *WiSARD Classifier*, the parameter M depends on the problem size.

In the classification phase, each test sample is given in input to all discriminators of the *WiSARD*. Multivariable datum representing a sample is encoded into a set of keys used to access all RAMs of all discriminators. Therefore, to classify each sample we have $N \times K$ accesses, where K is the number of classes for the problem. The K different responses from the discriminators are reduced by the argmax function (see Eq. (5)). Since the large part of the computation is spent in accessing RAMs, argmax reduction time is negligible and we can say that the classification time goes as $O(N \times K)$. Looking at Table 9 for datasets with a number of classes around 10, if we consider that the test set is 10% of the original dataset, we can

¹² It is not specified in the documentation.

Table 8

Pairwise comparisons among WNN and the other methods.

<i>p</i> -Value	Pairwise comparison of methods WNN vs							
	RF	MLP	GTB	SVC	LR	AB	kNN	GNB
z-value	9.26e−02	2.31e−01	3.86e−01	4.01e−01	3.21e+00	4.81e+00	4.88e+00	7.73e+00
Unadjusted <i>p</i>	9.26e−01	8.17e−01	7.00e−01	6.88e−01	1.33e−03	1.48e−06	1.08e−06	1.07e−14
<i>p</i> _{Nemenyi}	1.00e+00	1.00e+00	1.00e+00	1.00e+00	4.79e−02	5.32e−05	3.90e−05	3.84e−13
<i>p</i> _{Holm}	1.00e+00	1.00e+00	1.00e+00	1.00e+00	2.53e−02	4.14e−05	3.14e−05	3.73e−13
<i>p</i> _{Hochberg}	1.85e+00	4.08e+00	4.90e+00	5.51e+00	5.69e+00	5.69e+00	5.69e+00	5.69e+00
<i>p</i> _{Finner}	9.32e−01	8.52e−01	7.65e−01	7.65e−01	2.66e−03	5.91e−06	4.87e−06	1.92e−13
<i>p</i> _{Schaffer}	1.00e+00	1.00e+00	1.00e+00	1.00e+00	2.39e−02	4.14e−05	3.03e−05	2.98e−13

Table 9

Comparison of training and testing time of the five best methods (* = parallel version; best times in bold numbers; worse times in underlined numbers).

dataset (size:features:classes)	Methods									
	WNN		MLP		GTB		RF		SVC	
	Train	Test*	Train	Test	Train	Test	Train*	Test*	Train	Test
abalone (4174:10:28)	0.64	<u>0.43</u>	2.2	7.6e−04	<u>5.2</u>	0.0069	0.75	0.12	0.47	0.053
appendicitis (106:7:2)	0.016	0.02	0.1	1.3e−04	0.024	1.8e−04	<u>0.51</u>	0.11	4.5e−04	7.3e−05
bupa (345:6:2)	0.035	0.018	0.23	2.1e−04	0.029	2.3e−04	<u>0.28</u>	<u>0.15</u>	0.0024	2.1e−04
connect-41 (67557:126:3)	170	13	80	0.056	2.2000	0.26	5.8	0.24	<u>500</u>	<u>20</u>
ecoli (336:7:8)	0.13	0.033	0.37	2.5e−04	<u>0.64</u>	0.0011	0.29	<u>0.14</u>	0.0022	1.9e−04
flare (1066:38:6)	0.45	0.094	<u>0.71</u>	3.8e−04	0.42	7.7e−04	0.26	<u>0.14</u>	0.015	0.0016
ionosphere (351:33:2)	0.18	0.035	0.33	2e−04	0.056	2.4e−04	<u>0.55</u>	<u>0.11</u>	0.0027	2.6e−04
iris (150:4:3)	0.029	0.023	0.17	1.7e−04	0.099	3.5e−04	<u>0.28</u>	<u>0.14</u>	5.3e−04	7.7e−05
lymp (148:38:4)	0.093	0.035	0.24	2.5e−04	0.25	5.3e−04	<u>0.55</u>	<u>0.11</u>	9.9e−04	1e−04
magic (19020:10:2)	2.8	0.17	<u>16</u>	0.0063	2.6	0.0042	3.7	0.11	3.3	0.2
mushrooms (8124:112:2)	11	<u>0.64</u>	0.29	0.0046	<u>2.5</u>	0.0019	0.28	0.14	0.2	0.014
satimage (6435:36:6)	3.9	<u>0.77</u>	2	0.0015	<u>21</u>	0.018	1.2	0.11	0.28	0.046
splice (3175:60:2)	<u>8.9</u>	<u>1.6</u>	2.7	9.8e−04	1.6	0.0019	0.57	0.12	0.28	0.023
texture (5500:40:11)	3.3	<u>1.1</u>	3.3	9.7e−04	<u>17</u>	0.014	2	0.14	0.15	0.032
tic-tac-toe (958:27:2)	0.59	0.074	0.091	3.8e−04	0.29	7.2e−04	<u>0.83</u>	<u>0.13</u>	0.01	7.9e−04
wine (178:13:3)	0.062	0.0333	0.12	1.4e−04	0.12	3.3e−04	<u>0.32</u>	<u>0.14</u>	8.2e−04	9.6e−05
zoo (101:21:7)	0.061	0.037	0.016	1.5e−04	0.21	5.9e−04	<u>0.34</u>	<u>0.15</u>	0.0011	1.5e−04

understand why in these cases training and testing time are in the same order of magnitude.

Regarding the comparison between WNN timing measures and those of the other methods, we can say that, although classification times of our method are higher than the other methods, with respect to the training times, WNN performs better than MLP and RF (while similarly to GTB) in most of the examined datasets, in particular for large problem sizes (in term of number of features). This result is in agreement with the training formula of Eq. (3). The classification time is still too high, although when the problem size (number of features times the number of samples to test) is not so large, it is similar to the classification time of RF.

Considering the good accuracy and the overhead of our method, we might wonder in which case studies we could profitably use WiSARD as a classifier in real applications. The answer is quite articulate and we will try to provide it below.

In literature it is widely recognized that the training time of WiSARD is faster than in weighted ANNs. Nevertheless, from timing experiments reported in Table 9 the reader may notice that this is true only in few cases, while in the majority of datasets the training time of our method is close to that of MLP. This is an apparent contradiction that can be explained by reminding that WiSARD is very efficient in learning/recognizing binary patterns. On the contrary, in this work we used WiSARD as a classifier in the domain of multivariable real data. We did so because our aim was conducting an extensive comparison of WiSARD's accuracy with respect to state-of-the-art methods, when all these methods compete in a large set of classification problems in the domain of multivariable real data. To this purpose we adopted a uniform and parametric binarization scheme to transform input real vectors into binary patterns, regardless of the specific problem under study. Indeed, as discussed

before, the training and classification time strongly depend on the parameters of the binarization scheme.

When the purpose is different, such as developing an efficient classifier in a specific data domain, in literature there are many works proving how WiSARD provides solutions with very good results both in terms of accuracies and training times. As an example, mWANN-Tagger [36] is a speech recognition system based on WiSARD which adopts an efficient and optimized binarization scheme for textual data. The authors proved how mWANN-Tagger outperforms CRF (Conditional Random Fields) by two orders of magnitude in training time, while experiencing, like in our case, classification times higher than the competitors. In [37] ClusWiSARD, i.e. a clustering customization of WiSARD applied to two different credit analysis real-world problems, is proposed. In their work the authors demonstrate how ClusWiSARD outperforms SVM in accuracy, in training time (by 2 orders of magnitude), and it is slightly faster in testing time.

Fast training times of WiSARD is a great advantage when using this model in domains in which online learning is mandatory. In such domains, data become available in a sequential order and are used step-by-step to update (and improve) the classification capability of the model on future data, as opposed to batch learning techniques which generate the best predictor by learning on the entire training data set at once. As an example, mining data streams require control over learning as well as unlearning, ignoring past data according to some criteria. WiSARD training mechanism supports both knowledge expansion whenever it is required together with outdated knowledge discarding. This WiSARD's capability has been explored in [38,39] in two different applications scenarios, such as data stream clustering and video background online modeling.

10. Concluding remarks

WiSARD Classifier method is based on the *WiSARD* neural model of computation, which belongs to the class of weightless neural networks (WNNs). The main difference between traditional ANNs and WNNs is where the internal state of the neural network is stored and how it changes during the training phase and evolves towards a final one representing the trained model which is then used for classification. In ANNs the state is represented by the set of weighted connections, while in WNNs by the RAM neuron contents. Moreover, in ANNs the optimal weight configuration is reached by backpropagation (with or without pruning heuristics), while in the adopted WNN (i.e., the *WiSARD*) RAM contents are updated one time at each training sample (one-shot feedforward training). As a consequence, WNNs are faster than ANNs in the learning process. On contrary, learning and classification phases of WNNs could become slow when the network size grows, both in terms of RAM neuron number and number of cells in each neuron. Therefore, how to tune neuron parameters of WNNs is crucial in terms of performance.

Our proposal of a classification method based on *WiSARD* is not new, since in literature several classification/recognition systems have been proposed exploiting weightless neural networks (and *WiSARD* in particular) in several problem domains, mainly in the field of image processing. Thus, the main contribution of this work is not the method *WiSARD Classifier*, but a large experimental evaluation of its classification capabilities in terms of accuracy and timing in comparison to other methods from the state-of-the-art.

In this work we showed how *WiSARD Classifier*, when tested on a large ML dataset archive in comparison to several machine learners, ranked in the first four positions (at the 2nd place according to the Friedman ranking). From the statistical analysis of the experimental results that we have presented in this work, it is clear how our method is equivalent, in terms of the performance distribution across a large number of problems, to the following methods: Gradient Tree Boosting, Random Forest, and Multi-Layer Perceptron. In addition, pairwise comparisons of methods conducted in the statistical analysis proved that *WiSARD Classifier* is very close to Random Forest and Multi-Layer Perceptron, while it outperforms methods such as Support Vector Machines, Adaboost, kNN clustering and Gaussian Naive Bayes. This large experimental study demonstrated that, in the average, *WiSARD Classifier* finely competes with best ML methods when approaching many classification problems in very different domains. A weakness of our method is the processing time of the classification stage. In Section 9 we provided a comparison of timing measures among methods. As a general remark, our method performs better than Multi-Layer Perceptron and Random Forest while similarly to Gradient Tree Boosting in training time. On the other hand, classification time is higher than the benchmarking methods, although, in particular in the case of not-very large problems, it is similar to the classification time of Random Forest. We will further investigate optimization techniques to improve *WiSARD Classifier* performance, in terms of RAM neuron memorization strategy and in terms of data input encoding into the adopted binary retinas. The main advantage of *WiSARD Classifier* with regard to most of the considered (not only ANNs) methods is the possibility of online training which is typical of *WiSARD*. In this model the training is an incremental process: each sample adds knowledge to the neural network state, while either preserves or reinforces the previous acquired knowledge. Moreover this aspect allows adopting *WiSARD Classifier* in domains in which the learning stage is continuous and we want to use new training samples as soon as they are available in a non-batch modality.

In addition, *WiSARD Classifier* is the first ML classification method based on weightless neural networks and designed to be general-purpose since it exploits a uniform binarization scheme

for numeric multivariable data regardless on the number of data dimensions (features) and the problem domain. The result is a classifier able to process data in most common ML data formats (ARFF, LibSVM, CSV) with good classification scores, as the large experimental study discussed in this work has proved.

Appendix A. Introduction to statistical nonparametric tests

A description of the statistical tests is reported in the following subsections.

A.1 Friedman test

The Friedman test [40,41] is a multiple comparison test that aims to detect significant differences between the behavior of two or more algorithms. A very basic explanation of this statistical test for a case in which N problems (datasets) and k methods are considered is that the Friedman test ranks with r_i^j the result of method j ($1 \leq j \leq k$) on each problem i ($1 \leq i \leq N$) separately. The best performing method is assigned the rank of 1, the second best rank 2, and so on up to k . In case of tied values, the Friedman test assigns to each tied value the average of the ranks that would have been assigned without ties. Moreover, for each method j , it averages the ranks obtained in all problems: $R_j = \frac{1}{N} \sum_i r_i^j$. Under the null hypothesis, which states that all the chosen algorithms behave similarly (therefore their ranks R_j should be equal) the Friedman statistic S_F can be computed as:

$$S_F = \frac{12N}{k(k+1)} \left[\sum_j R_j^2 - \frac{k(k+1)^2}{4} \right] \quad (25)$$

which is distributed according to a χ^2 distribution with $k-1$ degrees of freedom, when N and k are big enough ($N > 10$ and $k > 5$ as a thumb rule, which holds in our case).

A.2 Friedman aligned rank test

In the Friedman Aligned Rank test [42] the average performance (in our case method accuracy) achieved by all methods in each problem is computed. Then, the difference between the performance obtained by a method and the average performance are computed. This step is repeated for each problem. The resulting matrix of differences d_i^j (aligned observations) are ranked between 1 and $k \cdot N$ relative to each other as they were independent samples. The ranks assigned to the aligned observations are called *aligned ranks*. The Friedman Aligned Rank statistic S_{AF} is defined as:

$$S_{AF} = \frac{(k-1) \left[\sum_{j=1}^k \hat{R}_j^2 - (kN^2/4)(kN+1)^2 \right]}{[kN(kN+1)(2kN+1)]/6 - (1/k) \sum_{i=1}^N \hat{R}_i^2} \quad (26)$$

where $\hat{R}_i$ is equal to the rank total of the i th problem and $\hat{R}_j$ is the rank total of the j th method. The statistic S_{AF} is distributed according to a χ^2 distribution with $k-1$ degrees of freedom.

A.3 Quade test

We also applied the Quade test [43] as multiple comparison test. Unlike the Friedman test, the Quade test takes into account the fact that some problems are more difficult than others, or that for some problems the performance differences are larger, while the Friedman test assumes all problems are equivalent in importance. Thus, the Quade test scales ranking of each problem depending on the differences observed in the methods' performances, obtaining a weighted ranking analysis of the results.

Table 10

Best parameters for methods LR, kNN, SVC and RF.

dataset (size:features)	LR		kNN		SVC			RF	
	C	penalty	algo	no neigh	C	gamma	kernel	n.estimators	criterion
abalone (4174:10)	1	l1	ballTree	30	1	1	rbf	200	gini
adult (48842:104)	1	l1	ballTree	30	1	0.001	linear	200	entropy
appendicitis (106:7)	0.001	l1	ballTree	3	1	0.001	linear	200	gini
australian (690:14)	1	l1	ballTree	30	0.1	1	rbf	200	gini
automobile (159:71)	1	l1	ballTree	10	1	0.001	linear	100	entropy
balance (625:4)	0.01	l2	brute	10	1	1	rbf	100	gini
banana (5300:2)	0.1	l2	ballTree	30	1	1	rbf	200	gini
bands (365:19)	1	l2	ballTree	5	1	1	rbf	100	gini
breast (263:9)	1	l1	ballTree	3	1	0.001	linear	100	entropy
bupa (345:6)	1	l1	ballTree	5	1	1	rbf	100	entropy
car (1728:21)	1	l1	ballTree	5	1	0.001	linear	300	entropy
chess-2 (3196:38)	1	l1	KDTree	10	1	1	rbf	100	gini
cleveland (297:13)	1	l1	ballTree	10	1	1	rbf	100	gini
coil2000 (9822:85)	1	l2	ballTree	10	0.001	0.001	linear	300	entropy
connect-41 (67557:126)	1	l1	brute	10	1	0.1	rbf	200	entropy
contraceptive (1473:9)	1	l1	ballTree	3	1	1	rbf	200	entropy
crx (653:42)	0.1	l1	ballTree	15	0.1	0.1	rbf	200	gini
dermatology (358:34)	1	l2	ballTree	10	1	0.1	rbf	100	gini
ecoli (336:7)	1	l1	ballTree	3	1	1	rbf	100	gini
flare (1066:38)	1	l2	brute	30	1	0.001	linear	100	gini
german (1000:24)	1	l2	ballTree	15	1	0.001	linear	100	entropy
glass (214:9)	1	l1	ballTree	5	1	1	rbf	200	entropy
haberman (306:3)	1	l1	ballTree	30	0.001	0.001	linear	300	entropy
hayes-roth (160:4)	1	l2	brute	10	1	1	rbf	100	entropy
heart (270:13)	1	l2	ballTree	10	1	1	rbf	300	gini
hepatitis (80:19)	0.001	l2	ballTree	3	0.001	0.001	linear	100	gini
housevotes (232:16)	1	l2	ballTree	3	0.1	0.001	linear	100	gini
ionosphere (351:33)	1	l1	ballTree	3	1	1	rbf	200	gini
iris (150:4)	1	l1	ballTree	3	1	0.001	linear	100	gini
led7digit (500:7)	1	l1	brute	15	1	0.001	linear	100	gini
letter (20000:16)	1	l1	brute	3	1	1	rbf	300	gini
lymp (148:38)	1	l2	ballTree	30	1	0.001	linear	200	entropy
magic (19020:10)	1	l1	ballTree	15	1	1	rbf	300	gini
mammographic (830:5)	1	l1	brute	30	0.1	0.001	linear	300	entropy
marketing (6876:13)	1	l1	ballTree	15	1	1	rbf	300	entropy
monk-2 (432:6)	1	l1	ballTree	3	1	1	rbf	100	entropy
movement-libras (360:90)	1	l2	ballTree	3	1	1	rbf	300	gini
mushrooms (8124:112)	1	l1	ballTree	3	1	0.001	linear	100	gini
newthyroid (215:5)	1	l1	ballTree	3	1	1	rbf	100	gini
nursery (12960:26)	0.1	l1	ballTree	10	1	0.1	rbf	300	entropy
optdigits (5620:64)	1	l2	ballTree	5	1	0.1	rbf	200	gini
page-blocks (5472:10)	1	l1	brute	3	1	1	rbf	100	gini
penbased (10992:16)	1	l1	ballTree	3	1	1	rbf	300	gini
phomene (5404:5)	1	l2	ballTree	3	1	1	rbf	200	entropy
pima (768:8)	1	l1	ballTree	30	1	1	rbf	300	gini
post-operative (87:21)	0.1	l2	ballTree	10	0.001	0.001	linear	200	gini
ring (7400:20)	1	l2	ballTree	3	1	1	rbf	200	entropy
saheart (462:9)	1	l1	ballTree	30	1	1	rbf	100	entropy
satimage (6435:36)	1	l1	ballTree	3	1	1	rbf	300	entropy
segment (2310:19)	1	l1	ballTree	3	1	1	rbf	200	entropy
shuttle (57999:9)	1	l1	ballTree	3	1	1	rbf	100	gini
sonar (208:60)	1	l1	ballTree	3	1	1	rbf	100	entropy
spambase (4597:57)	1	l1	ballTree	5	1	1	rbf	300	entropy
spectfheart (267:44)	0.001	l2	ballTree	30	0.001	0.001	linear	100	gini
splice (3175:60)	1	l1	brute	5	1	0.1	rbf	200	gini
tae (151:5)	0.001	l2	ballTree	5	0.001	0.001	linear	200	gini
texture (5500:40)	1	l1	ballTree	3	1	1	rbf	300	entropy
thyroid (7200:21)	1	l1	ballTree	5	1	0.001	linear	100	entropy
tic-tac-toe (958:27)	1	l1	brute	5	1	0.001	linear	300	gini
titanic (2201:3)	1	l1	brute	5	1	1	rbf	100	gini
twonorm (7400:20)	0.1	l1	ballTree	30	1	1	rbf	300	gini
vehicle (846:18)	1	l1	ballTree	3	1	1	rbf	100	gini
vowel (990:13)	1	l1	ballTree	3	1	1	rbf	200	entropy
wdbc (569:30)	1	l2	ballTree	3	1	0.001	linear	200	entropy
wine (178:13)	1	l1	ballTree	10	1	0.001	linear	100	gini
winequality-red (1599:11)	1	l2	ballTree	5	1	1	rbf	100	entropy
winequality-white (4898:11)	1	l1	ballTree	15	1	1	rbf	200	gini
wisconsin (683:9)	0.001	l2	ballTree	5	1	0.001	linear	100	gini
yeast (1484:8)	1	l1	ballTree	10	1	1	rbf	100	gini
zoo (101:21)	1	l1	ballTree	5	1	0.001	linear	100	gini

Table 11
Best parameters for methods GTB, MLP, AB and WNN.

dataset (size:features)	GTB		MLP		AB		WNN	
	n.estimators	max depth	size	solver	n.estimators	algo	n.tics	n.bits
abalone (4174:10)	100	2	200	adam	20	SAMME	4096	16
adult (48842:104)	200	5	100	adam	100	SAMME.R	1024	16
appendicitis (106:7)	100	2	200	adam	50	SAMME	1024	8
australian (690:14)	100	3	100	adam	20	SAMME	4096	16
automobile (159:71)	100	3	100	adam	50	SAMME	1024	16
balance (625:4)	100	2	100	lbfgs	50	SAMME.R	512	4
banana (5300:2)	300	3	300	lbfgs	100	SAMME	1024	16
bands (365:19)	100	3	100	adam	20	SAMME	1024	8
breast (263:9)	100	2	200	adam	20	SAMME	2048	8
bupa (345:6)	100	2	300	adam	100	SAMME	2048	16
car (1728:21)	300	3	100	adam	100	SAMME	256	16
chess-2 (3196:38)	300	5	200	adam	100	SAMME.R	4096	16
cleveland (297:13)	100	2	200	lbfgs	20	SAMME	128	2
coil2000 (9822:85)	100	3	100	adam	20	SAMME	256	4
connect-41 (67557:126)	300	10	300	adam	100	SAMME.R	1024	16
contraceptive (1473:9)	100	2	200	adam	20	SAMME.R	4096	16
crx (653:42)	100	3	100	adam	20	SAMME	1024	16
dermatology (358:34)	200	2	100	lbfgs	20	SAMME.R	256	8
ecoli (336:7)	200	3	300	adam	100	SAMME	2048	16
flare (1066:38)	100	2	200	lbfgs	20	SAMME	512	4
german (1000:24)	100	5	300	adam	50	SAMME	1024	16
glass (214:9)	200	2	300	adam	50	SAMME	1024	32
haberman (306:3)	100	2	100	adam	20	SAMME	512	8
hayes-roth (160:4)	100	2	200	lbfgs	20	SAMME.R	512	8
heart (270:13)	200	2	100	adam	20	SAMME	2048	16
hepatitis (80:19)	100	2	100	lbfgs	50	SAMME.R	256	8
housevotes (232:16)	100	2	100	lbfgs	50	SAMME.R	256	4
ionosphere (351:33)	100	2	200	adam	50	SAMME	256	8
iris (150:4)	100	5	300	adam	50	SAMME.R	1024	16
led7digit (500:7)	100	2	100	adam	100	SAMME	1024	16
letter (20000:16)	300	5	300	lbfgs	100	SAMME	2048	16
lymp (148:38)	200	2	300	adam	20	SAMME.R	512	8
magic (19020:10)	200	3	300	lbfgs	100	SAMME.R	1024	16
mammographic (830:5)	100	2	200	lbfgs	50	SAMME	512	4
marketing (6876:13)	100	2	100	adam	100	SAMME	1024	16
monk-2 (432:6)	100	2	300	lbfgs	50	SAMME	64	4
movement-libras (360:90)	100	2	200	adam	100	SAMME	1024	16
mushrooms (8124:112)	300	2	100	lbfgs	50	SAMME.R	1024	16
newthyroid (215:5)	100	10	200	lbfgs	20	SAMME	512	32
nursery (12960:26)	300	5	200	adam	50	SAMME	4096	16
optdigits (5620:64)	200	3	200	adam	100	SAMME	1024	16
page-blocks (5472:10)	100	2	200	lbfgs	100	SAMME	256	32
penbased (10992:16)	300	3	300	lbfgs	100	SAMME	256	32
phomene (5404:5)	200	10	300	lbfgs	100	SAMME.R	1024	32
pima (768:8)	100	3	100	adam	50	SAMME	4096	16
post-operative (87:21)	200	3	200	lbfgs	20	SAMME	256	4
ring (7400:20)	300	2	200	lbfgs	100	SAMME.R	4096	16
saheart (462:9)	100	10	300	adam	50	SAMME	4096	32
satimage (6435:36)	300	5	100	lbfgs	50	SAMME	128	32
segment (2310:19)	200	3	100	lbfgs	100	SAMME	128	32
shuttle (57999:9)	100	2	200	lbfgs	100	SAMME	512	32
sonar (208:60)	100	3	100	adam	100	SAMME.R	4096	16
spambase (4597:57)	200	5	100	adam	100	SAMME.R	1024	16
spectfheart (267:44)	100	2	100	adam	100	SAMME	4096	32
splice (3175:60)	300	3	200	adam	50	SAMME	1024	16
tae (151:5)	100	3	100	lbfgs	50	SAMME	8192	32
texture (5500:40)	300	2	200	lbfgs	100	SAMME	4096	32
thyroid (7200:21)	200	2	200	lbfgs	20	SAMME.R	1024	16
tic-tac-toe (958:27)	300	3	200	lbfgs	100	SAMME.R	8192	32
titanic (2201:3)	300	3	100	lbfgs	20	SAMME	256	4
twonorm (7400:20)	300	2	200	adam	100	SAMME.R	4096	32
vehicle (846:18)	300	2	100	lbfgs	100	SAMME	1024	16
vowel (990:13)	100	3	200	lbfgs	100	SAMME	512	8
wdbc (569:30)	200	2	100	lbfgs	100	SAMME.R	1024	32
wine (178:13)	100	3	100	adam	50	SAMME	2048	16
winequality-red (1599:11)	200	3	200	adam	20	SAMME	8192	32
winequality-white (4898:11)	100	5	300	lbfgs	100	SAMME	8192	32
wisconsin (683:9)	200	2	200	adam	20	SAMME	4096	16
yeast (1484:8)	100	2	200	adam	50	SAMME	2048	16
zoo (101:21)	100	2	100	lbfgs	20	SAMME	128	4

In the Quade test, at first ranks r_i^j are computed as in the Friedman test. Secondly, the original observed performance of method j on problem i , namely x_{ij} , is used to compute the performance range of methods on each dataset i , namely δ_i :

$$\delta_i = \max_j x_{ij} - \min_j x_{ij}. \quad (27)$$

Then, each dataset i is associated with a rank, namely Q_i , which is 1 for the dataset with lowest range, 2 for the second lowest, and so on to the dataset with highest range, which gets rank n . Let $Q_1, Q_2, \dots, Q_n$ be the ranks assigned to dataset 1, 2, $\dots$, n , respectively, then the statistic:

$$s_i^j = Q_i \left[r_i^j - \frac{k+1}{2} \right] \quad (28)$$

represents the relative size of each observed performance within the dataset, adjusted to reflect the relative significance of the dataset in which it appears. The final formula for Quade test is given in Eq. (29), where $S_j = \sum_{i=1}^N s_i^j$. The S_Q statistic is distributed according to the F distribution with $k-1$ and $(k-1)(n-1)$ degrees of freedom:

$$S_Q = \frac{(n-1) \frac{1}{n} \sum_{j=1}^k S_j^2}{n(n+1)(2n+1)(k)(k+1)(k-1)/72 - \frac{1}{n} \sum_{j=1}^k S_j^2}. \quad (29)$$

A.4 Contrast estimation based on medians

Contrast estimation is a useful procedure to evaluate pairwise the difference in performance of various algorithms over multiple experiments. This procedure assumes that the expected differences between performances of algorithms are the same across data sets. Consequently, we are interested in estimating the contrast between medians of samples of results considering all pairwise comparisons. It obtains a quantitative difference computed through medians between two algorithms over multiple data sets, but the value obtained will change when using other data sets in the experiment. It works as follows.

For each pair of k methods in the experiment, it computes the difference between the performances of the two methods, namely u and v in each problem i of the N datasets:

$$D_{i(uv)} = x_{iu} - x_{iv} \quad (30)$$

where $u, v \in [1, \dots, k]$ and $i \in [1, \dots, N]$. For each pair of methods we find the median of differences $D_{i(uv)}$ across all i datasets, and we call it Z_{uv} . Note that for each pair $u, v \in [1, \dots, k]$ $Z_{uv} = Z_{vu}$ and $Z_{uu} = 0$. Then we compute for each method u the mean:

$$m_u = \frac{\sum_{j=1}^k Z_{uj}}{k}. \quad (31)$$

Finally the contrast estimator of each couple of methods u, v is $CE_{u,v} = m_u - m_v$. Note that $CE_{u,v}$ is either positive or negative if method u performed better or worse than method v . The contrast estimation matrix is formed by the elements CE_{ij} .

Appendix B. Optimal parameters configuration for experiments

In this appendix we report two tables (Tables 10 and 11) which summarize results of the model selection process we carried out to find the best hyper-parameters for all methods used in the experiments.

Best configurations of parameters has been carried out by exhaustively search in a parameter grid to generate candidates by means of a five-fold cross-validation by means of the GridSearchCV utility of Scikit Learn library. For each method and for

each dataset we decided to evaluate best values for a small set of parameters (two or three, depending on the method) by varying their values in the ranges which are highlighted in bold in Table 11. You may notice that method GNB does not appear in the tables, since its implementation simply has no parameter that can be set by the programmer.

As we can see in the tables, for each method hyper-parameters settings were different depending on the target classification problem size and specificity. Once parameter configurations were identified, they were used in the experiments (see Section 5) we performed to evaluate and compare average accuracy of methods.

References

- [1] W.W. Bledsoe, I. Browning, Pattern recognition and reading by machine, in: Papers Presented at the Eastern Joint IRE-AIEE-ACM Computer Conference, IRE-AIEE-ACM '59 (Eastern), ACM, New York, NY, USA, 1959, pp. 225–232, <http://dx.doi.org/10.1145/1460299.1460326>.
- [2] I. Aleksander, M. De Gregorio, F.M.G. França, P.M.V. Lima, H. Morton, A brief introduction to weightless neural systems, Proc. of 17th ESANN (2009) 299–305.
- [3] T.B. Ludermir, A.C. Carvalho, A.P. Braga, M.C.P. Souto, Weightless neural models: a review of current and past works, Neural Comput. Surv. 2 (1999) 41–61.
- [4] P. Kanerva, Sparse Distributed Memory, MIT Press, Cambridge, MA, USA, 1988.
- [5] J.S. Albus, A theory of cerebellar function, Math. Biosci. 10 (1) (1971) 25–61, [http://dx.doi.org/10.1016/0025-5564\(71\)90051-4](http://dx.doi.org/10.1016/0025-5564(71)90051-4).
- [6] V.M.G. Ferreira, F.M.G. França, Weightless implementations of weighted neural networks, Proc. of IV Simpósio Brasileiro de Redes Neurais (1997) 53–54.
- [7] I. Aleksander, Microcircuit learning nets: Hamming-distance behaviour, Electron. Lett. 6 (5) (1970) 134–136, <http://dx.doi.org/10.1049/el:19700092>.
- [8] A.F.D. Souza, F. Pedroni, E. Oliveira, P.M. Ciarelli, W.F. Henrique, L. Veronese, C. Badue, Automated multi-label text categorization with VG-RAM weightless neural networks, Neurocomputing 72 (2009) 2209–2217, <http://dx.doi.org/10.1016/j.neucom.2008.06.028>.
- [9] C. Souza, F. Nobre, P.M.V. Lima, R. Silva, R. Brindeiro, F.M.G. França, Recognition of HIV-1 subtypes and antiretroviral drug resistance using weightless neural networks, Proc. of ESANN'12 (2012) 429–434.
- [10] H.C.C. Carneiro, F.M.G. França, P.M.V. Lima, WANN-tagger – a weightless artificial neural network tagger for the Portuguese language, in: J. Filipe, J. Kacprzyk (Eds.), Proc. of IJCCI (ICFC-ICNC), SciTePress, 2010, pp. 330–335.
- [11] D.O. Cardoso, J. Gama, M. De Gregorio, F.M.G. França, M. Giordano, P.M.V. Lima, WIPS: the WiSARD indoor positioning system, Proc. of ESANN'12 (2012) 521–526.
- [12] I. Aleksander, T.J. Stonham, Guide to pattern recognition using random access memories, IEE J. Comput. Digit. Tech. 2 (1) (1979) 29–40, <http://dx.doi.org/10.1049/ij-cdt.1979.0009>.
- [13] I. Aleksander, H. Morton, An Introduction to Neural Computing, Chapman & Hall, London, 1990.
- [14] I. Aleksander, T.J. Stonham, Guide to pattern recognition using random access memories, IEE J. Comput. Digit. Tech. 2 (1) (1979) 29–40, <http://dx.doi.org/10.1049/ij-cdt.1979.0009>.
- [15] T.J. Stonham, Improved Hamming-distance analysis for digital learning networks, Electron. Lett. 13 (6) (1977) 155–156, <http://dx.doi.org/10.1049/el:19770110>.
- [16] N.M. Allinson, A.R. Kolcz, N-Tuple Neural Networks, Springer US, Boston, MA, 1997, pp. 3–14, http://dx.doi.org/10.1007/978-1-4615-6099-9_1.
- [17] I. Aleksander, Microcircuit learning nets: Hamming-distance behaviour, Electron. Lett. 6 (5) (1970) 134–136, <http://dx.doi.org/10.1049/el:19700092>.
- [18] G.P. Steck, Stochastic model for the Browning–Bledsoe pattern recognition scheme, IRE Trans. Electron. Comput. EC-11 2 (1962) 274–282, <http://dx.doi.org/10.1109/TEC.1962.5219360>.
- [19] W.W. Bledsoe, C.L. Bisson, Improved memory matrices for the n-tuple pattern recognition method, IRE Trans. Electron. Comput. EC-11 3 (1962) 414–415, <http://dx.doi.org/10.1109/IRETELC.1962.5407930>.
- [20] J.R. Ullmann, Experiments with the n-tuple method of pattern recognition, IEEE Trans. Comput. C-18 12 (1969) 1135–1137, <http://dx.doi.org/10.1109/T-C.1969.222599>.
- [21] B.P.A. Grieco, P.M.V. Lima, M. De Gregorio, F.M. França, Producing pattern examples from “mental” images, Neurocomputing 73 (7–9) (2010) 1057–1064, <http://dx.doi.org/10.1016/j.neucom.2009.11.015>.
- [22] M. De Gregorio, M. Giordano, Apoptosis and neurogenesis in DRASIW-like systems, Neurocomputing 183 (2016) 3–12, <http://dx.doi.org/10.1016/j.neucom.2015.05.137>.
- [23] M. De Gregorio, M. Giordano, Cloning DRASIW systems via memory transfer, Neurocomputing 192 (2016) 115–127, <http://dx.doi.org/10.1016/j.neucom.2016.01.087>.
- [24] M. De Gregorio, M. Giordano, Background estimation by weightless neural networks, Pattern Recognit. Lett. 96 (2017) 55–65, <http://dx.doi.org/10.1016/j.patrec.2017.05.029>.

- [25] M. De Gregorio, M. Giordano, S. Rossi, M. Staffa, Experimenting WNN support in object tracking systems, *Neurocomputing* 183 (2016) 79–89, <http://dx.doi.org/10.1016/j.neucom.2015.09.117>.
- [26] D. Broomhead, D. Lowe, Multivariable functional interpolation and adaptive networks, *Complex Syst.* 2 (1988) 321–355.
- [27] G.D. Tattersall, S. Foster, R.D. Johnston, Single-layer lookup perceptrons, *IEE Proc. F – Radar Signal Process.* 138 (1) (1991) 46–54, <http://dx.doi.org/10.1049/ip-f-2.1991.0008>.
- [28] F. Schwenker, H.A. Kestler, G. Palm, Three learning phases for radial basis-function networks, *Neural Netw.* 14 (4) (2001) 439–458, [http://dx.doi.org/10.1016/S0893-6080\(01\)00027-2](http://dx.doi.org/10.1016/S0893-6080(01)00027-2).
- [29] C. Chang, C. Lin, LIBSVM: a library for support vector machines, *ACM Trans. Intell. Syst. Technol.* 2 (2011) 1–27.
- [30] I. Aleksander, R.C. Albrow, Pattern recognition with adaptive logic elements, *Proceedings of the IEE-NPL Conference on Pattern Recognition* (1968) 68–74.
- [31] Z.-H. Zhou, *Ensemble Methods: Foundations and Algorithms*, 1st ed., Chapman & Hall/CRC, 2012.
- [32] J.H. Friedman, Greedy function approximation: a gradient boosting machine, *Ann. Stat.* 29 (5) (2001) 1189–1232 <http://www.jstor.org/stable/2699986>.
- [33] C. Gini, *Variabilità e mutabilità: contributo allo studio delle distribuzioni delle relazioni statistiche*, Paolo Cuppin Publisher, 1912.
- [34] J. Demšar, Statistical comparisons of classifiers over multiple data sets, *J. Mach. Learn. Res.* 7 (2006) 1–30 <http://dl.acm.org/citation.cfm?id=1248547.1248548>.
- [35] J. Derrac, S. García, D. Molina, F. Herrera, A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms, *Swarm Evol. Comput.* 1 (1) (2011) 3–18, <http://dx.doi.org/10.1016/j.swevo.2011.02.002>.
- [36] H.C. Carneiro, F.M. França, P.M. Lima, Multilingual part-of-speech tagging with weightless neural networks, *Neural Netw.* 66 (2015) 11–21, <http://dx.doi.org/10.1016/j.neunet.2015.02.012>.
- [37] D.O. Cardoso, D.S. Carvalho, D.S. Alves, D.F. Souza, H.C. Carneiro, C.E. Pedreira, P.M. Lima, F.M. França, Financial credit analysis via a clustering weightless neural classifier, *Neurocomputing* 183 (2016) 70–78, <http://dx.doi.org/10.1016/j.neucom.2015.06.105>.
- [38] D.O. Cardoso, F.M.G. França, J. Gama, WCDS: a two-phase weightless neural system for data stream clustering, *New Gener. Comput.* 35 (4) (2017) 391–416, <http://dx.doi.org/10.1007/s00354-017-0018-y>.
- [39] M.D. De Gregorio, M. Giordano, Background estimation by weightless neural networks, *Pattern Recognit. Lett.* 96 (2017) 55–65, <http://dx.doi.org/10.1016/j.patrec.2017.05.029>.
- [40] M. Friedman, The use of ranks to avoid the assumption of normality implicit in the analysis of variance, *J. Am. Stat. Assoc.* 32 (200) (1937) 675–701 <http://www.jstor.org/stable/2279372>.
- [41] M. Friedman, A correction: the use of ranks to avoid the assumption of normality implicit in the analysis of variance, *J. Am. Stat. Assoc.* 34 (205) (1939) 109 <http://www.jstor.org/stable/2279169>.
- [42] J. Hodges, E. Lehmann, Rank methods for combination of independent experiments in analysis of variance, in: J. Rojo (Ed.), *Selected Works of E. L. Lehmann, Selected Works in Probability and Statistics*, Springer US, 2012, pp. 403–418, http://dx.doi.org/10.1007/978-1-4614-1412-4_35.
- [43] D. Quade, Using weighted rankings in the analysis of complete blocks with additive block effects, *J. Am. Stat. Assoc.* 74 (367) (1979) 680–683 <http://www.jstor.org/stable/2286991>.